HAUTE ÉCOLE
D'INGÉNIERIE ET DE GESTION
DU CANTON DE VAUD
www.heig-vd.ch

Département des Technologies de l'information et de la communication (TIC)
Filière Télécommunications
Orientation Sécurité de l'information

Travail de Bachelor

Support graphique dans SO3

Étudiant	**Nikolaos Garanis**
Enseignant responsable	Prof. Daniel Rossier
Année académique	2019 – 2020

Yverdon-les-Bains, le 31 juillet 2020

Département des Technologies de l'information et de la communication (TIC)
Filière Télécommunications
Orientation Sécurité de l'information
Étudiant : Nikolaos Garanis
Enseignant responsable : Prof. Daniel Rossier

Travail de Bachelor 2019 – 2020

Support graphique dans SO3

Résumé publiable

La technologie *Smart Object Oriented* (SOO), mise au point par l'institut REDS de l'HEIG–VD, permet la migration de systèmes d'exploitation (OS) en cours d'exécution parmi un ensemble interconnectés de systèmes embarqués. Cette technologie introduit la notion de *Smart Object*, qui fait référence à un système embarqué tel qu'une Raspberry Pi 4, et celle de *Mobile Entity* (ME), qui représente l'OS migrant en cours d'exécution.

La technologie SOO s'inscrit notamment dans le contexte de la domotique — ensemble de techniques permettant le contrôle, grâce à l'informatique, d'installations présentes dans un bâtiment. La possibilité pour une ME de présenter une interface graphique à l'utilisateur — par exemple à travers un écran tactile — ouvre la porte à de nombreux types d'applications. Une telle interface permettrait à l'utilisateur de visualiser les données gérées par la ME (e.g. données météorologiques) mais aussi d'interagir avec la ME (e.g. modifications de paramètres, visualisation de données sous un autre format).

Ainsi, l'objectif de ce travail de Bachelor est l'ajout du support graphique et de périphériques d'entrée dans SO3, OS développé par le REDS et utilisé comme base des ME. Une librairie graphique existante devra être intégrée dans SO3, d'abord lorsque celui-ci s'exécute de manière autonome. Puis, il s'agira de créer une ME basée sur cette nouvelle version de SO3 et de l'intégrer dans le framework SOO. Le développement se fera initialement dans un environnement émulé, puis il devra être déployé sur une Raspberry Pi 4.

Étudiant :	Date et lieu :	Signature :
Nikolaos Garanis	.....	.....
Enseignant responsable :	Date et lieu :	Signature :
Prof. Daniel Rossier	.....	.....

Préambule

Ce travail de Bachelor (ci-après TB) est réalisé en fin de cursus d'études, en vue de l'obtention du titre de Bachelor of Science HES-SO en Ingénierie.

En tant que travail académique, son contenu, sans préjuger de sa valeur, n'engage ni la responsabilité de l'auteur, ni celles du jury du travail de Bachelor et de l'École.

Toute utilisation, même partielle, de ce TB doit être faite dans le respect du droit d'auteur.

HEIG-VD

Vincent Peiris
Chef de département TIC

Yverdon-les-Bains, le 31 juillet 2020

Authentification

Le soussigné, Nikolaos Garanis, atteste par la présente avoir réalisé ce travail et n'avoir utilisé aucune autre source que celles expressément mentionnées.

Nikolaos Garanis

Yverdon-les-Bains, le 31 juillet 2020

Cahier des charges

Résumé du problème

La technologie *Smart Object Oriented* (SOO), mise au point par l'institut REDS de l'HEIG–VD, permet la migration de systèmes d'exploitation (OS) en cours d'exécution parmi un ensemble interconnectés de systèmes embarqués. Cette technologie introduit la notion de *Smart Object*, qui fait référence à un système embarqué tel qu'une Raspberry Pi 4, et celle de *Mobile Entity* (ME), qui représente l'OS migrant en cours d'exécution.

La technologie SOO s'inscrit notamment dans le contexte de la domotique — ensemble de techniques permettant le contrôle, grâce à l'informatique, d'installations présentes dans un bâtiment. La possibilité pour une ME de présenter une interface graphique à l'utilisateur — par exemple à travers un écran tactile — ouvre la porte à de nombreux types d'applications. Une telle interface permettrait à l'utilisateur de visualiser les données gérées par la ME (e.g. données météorologiques) mais aussi d'interagir avec la ME (e.g. modifications de paramètres, visualisation de données sous un autre format).

Ainsi, l'objectif de ce travail de Bachelor est l'ajout du support graphique et de périphériques d'entrée dans SO3, OS développé par le REDS et utilisé comme base des ME. Une librairie graphique existante devra être intégrée dans SO3, d'abord lorsque celui-ci s'exécute de manière autonome. Puis, il s'agira de créer une ME basée sur cette nouvelle version de SO3 et de l'intégrer dans le framework SOO. Le développement se fera initialement dans un environnement émulé, puis il devra être déployé sur une Raspberry Pi 4.

Cahier des charges

SO3

Le développement aura lieu sur la plateforme ARM Versatile Express qui sera émulée par le logiciel QEMU.

Librairie graphique

Une librairie graphique existante doit être choisie et placée dans l'espace utilisateur de SO3. La librairie nécessite d'être intégrée au système de compilation de SO3 afin que les applications de cet OS puissent l'utiliser.

Une application de démonstration doit être développée afin de tester le bon fonctionnement du développement effectué au cours de ce TB ainsi que de montrer les possibilités offertes par la librairie (e.g. éléments graphiques).

Écriture de drivers

Framebuffer Il s'agit ici de trouver sur la plateforme émulée un contrôleur mettant à disposition un framebuffer et d'écrire un driver permettant à SO3 de reconnaître et d'utiliser ce contrôleur afin de pouvoir afficher une interface graphique à l'écran.

Périphériques d'entrée À nouveau, il faut d'abord trouver le où les contrôleurs permettant de communiquer avec le clavier et la souris. Puis, les drivers doivent être écrits. Ils permettront la récupération des événements générés par le clavier et la souris.

Adaptation de SO3

Afin de pouvoir contrôler et interroger ces drivers, il est requis d'adapter et de créer certains appels système (noyau SO3). Par exemple, il faut pouvoir écrire dans l'espace mémoire dédié au framebuffer ou encore récupérer les coordonnées de la souris.

Il est aussi demandé que cette communication avec les drivers puisse se faire depuis une application SO3. Les appels systèmes modifiés doivent pouvoir être appelés depuis l'espace utilisateur.

SOO

Cette seconde partie vise à créer une ME basée sur SO3 et pouvant être utilisée par SOO. Cette ME doit intégrer les changements effectués lors de la première partie. Le développement aura lieu sur la plateforme ARM Versatile Express émulée par QEMU mais il s'agira aussi de déployer SOO et la ME sur une Raspberry Pi 4. Cette partie se compose de plusieurs étapes.

Développement des interfaces virtuelles

Les ME n'ont pas un accès direct au matériel. C'est SOO qui s'occupe de faire l'intermédiaire entre la ME et le matériel par le biais d'interfaces virtuelles. Ces interfaces nécessitent un développement autant du côté de la ME (SO3) que du côté de SOO.

Il faut donc développer deux interfaces virtuelles, une pour le framebuffer et une autre pour la gestion des entrées utilisateur (e.g. clavier, souris, écran tactile).

Portage sur Raspberry Pi 4

Les modifications apportées à la ME et à SOO doivent être portées sur la Pi 4 lorsque cela est nécessaire. Bien que la Pi 4 ait un processeur ARM, son architecture est très différente de la plateforme Versatile Express et possède d'autres contrôleurs.

Afin de tester l'intégration de la libraire graphique et le support des périphériques d'entrée, plusieurs écrans tactiles seront choisis et proposés à l'achat au REDS. Les écrans devront être testés avec la nouvelle ME. Le support des événements tactiles devra aussi être ajouté.

Table des matières

Préambule v

Authentification vii

Cahier des charges ix

1 Introduction 1

2 Interface graphique 3

2.1 Choix de LittlevGL . . . 3

2.2 Intégration et configuration . . . 4

2.3 Compilation . . . 5

2.4 Utilisation et fonctionnement . . . 6

2.4.1 Affichage de l'interface . . . 7

2.4.1.1 Description et utilisation du framebuffer . . . 7

2.4.1.2 Driver d'affichage LittlevGL . . . 8

2.4.2 Gestion des périphériques d'entrée . . . 9

2.4.2.1 Driver d'entrée – Souris . . . 10

2.4.2.2 Driver d'entrée – Clavier . . . 11

2.4.3 Accès au système de fichiers . . . 11

2.4.4 Application de démonstration . . . 13

3 SO3 15

3.1 Environnement de développement et d'exécution . . . 15
3.1.1 QEMU . . . 15
3.1.2 ARM Versatile Express . . . 16
3.2 Fichiers spéciaux . . . 16
3.2.1 Virtual Filesystem . . . 17
3.2.2 Solution implémentée . . . 18
3.3 Driver du framebuffer . . . 20
3.3.1 Description du PL111 . . . 20
3.3.1.1 Activation . . . 21
3.3.1.2 Position du framebuffer . . . 21
3.3.1.3 Définition de la résolution . . . 23
3.3.1.4 Registre de contrôle . . . 23
3.3.1.5 Écriture des registres . . . 24
3.3.2 Mapping de la mémoire . . . 24
3.3.3 Propriétés du framebuffer . . . 25
3.4 Drivers du clavier et de la souris . . . 26
3.4.1 Description du PL050 . . . 27
3.4.1.1 Activation . . . 27
3.4.1.2 Envoi de données . . . 27
3.4.1.3 Réception de données . . . 28
3.4.1.4 Traitement des paquets . . . 29
3.4.2 Coordonnées et touches appuyées . . . 31

4 SOO **33**

4.1 Introduction . . . 33
4.2 Gestion de plusieurs ME . . . 34
4.3 Gestion des framebuffers . . . 36
4.3.1 Création du backend et frontend . . . 36
4.3.2 Emplacement du framebuffer . . . 38

4.3.3 Taille du framebuffer . . . 39
4.3.3.1 Obtention des propriétés du framebuffer . . . 39
4.3.3.2 Transmission des propriétés à la ME . . . 41
4.3.4 Réservation de l'espace mémoire . . . 43
4.3.4.1 Création du driver virtuel . . . 43
4.3.5 Mapping du framebuffer dans l'agence . . . 44
4.3.6 Affichage des différents framebuffers . . . 45
4.3.7 Notifications extérieures . . . 47
4.3.8 Terminaison d'une ME . . . 48
4.4 Gestion des périphériques d'entrée . . . 49
4.4.1 Événements d'entrée . . . 49
4.4.2 Création du backend et frontend . . . 50
4.4.3 Gestion des ME . . . 50
4.4.4 Transmission des événements . . . 51
4.4.4.1 Du backend au frontend . . . 51
4.4.4.2 Du frontend aux drivers virtuels . . . 52
4.5 Portage sur Raspberry Pi 4 . . . 54
4.5.1 Introduction . . . 54
4.5.2 Écrans tactiles . . . 54
4.5.3 Gestion des événements tactiles . . . 55
4.5.3.1 Fonctionnement avec Raspbian . . . 55
4.5.3.2 Fonctionnement avec SOO . . . 55
4.5.3.3 Implémentation du support des événements tactiles . . . 56
4.5.4 Gestion des framebuffers . . . 57
4.5.4.1 Détection de la résolution . . . 57
4.5.4.2 Création du framebuffer . . . 58
4.5.4.3 Affichage des différents framebuffers – Adaptation . . . 59

5 Conclusion **61**

Bibliographie **63**

Chapitre 1

Introduction

L'institut REDS [1] de l'HEIG–VD a mis au point la technologie *Smart Object Oriented* (SOO) [26] [21] permettant la migration d'environnements en cours d'exécution parmi un ensemble interconnectés de systèmes embarqués. Dans ce contexte, la notion de *Smart Object* représente un système embarqué tel qu'une Raspberry Pi 4 et celle de *Mobile Entity* (ME) se réfère à l'environnement migrant en cours d'exécution.

SOO se présente sous la forme d'un framework conçu autour du noyau Linux et d'un hyperviseur basé sur les concepts XEN de virtualisation pour systèmes embarqués. Une Mobile Entity est un système d'exploitation (OS) virtualisé par cet hyperviseur. C'est dans ce cadre que SO3 [20] (*Smart Object Oriented Operating system*) a été développé à partir de zéro par le REDS, fournissant ainsi un OS léger et facilement modifiable.

La technologie SOO s'inscrit notamment dans le contexte de la domotique — ensemble de techniques permettant le contrôle, grâce à l'informatique, d'installations présentes dans un bâtiment. La possibilité pour une ME de présenter une interface graphique à l'utilisateur — par exemple à travers un écran tactile — ouvre la porte à de nombreux types d'applications. Une telle interface permettrait à l'utilisateur de visualiser les données gérées par la ME (e.g. données météorologiques) mais aussi d'interagir avec la ME (e.g. modifications de paramètres, visualisation de données sous un autre format).

L'objectif de ce travail de Bachelor est donc l'ajout du support graphique dans SO3. Cela passe par le choix et l'intégration d'une librairie graphique existante dans SO3 et non pas par l'implémentation depuis zéro d'une telle librairie. Ce travail se déroule en deux étapes. Premièrement, il s'agit d'implémenter le support graphique lorsque SO3 s'exécute de manière autonome (i.e. en dehors du framework SOO). Il est aussi nécessaire de supporter des périphériques d'entrée comme le clavier et la souris afin d'interagir avec l'interface graphique.

Deuxièmement, il s'agit de faire fonctionner cette interface graphique dans une ME basée

1. Reconfigurable & embedded Digital Systems

sur SO3 avec les modifications susmentionnées. Lorsque la ME est virtualisée, la communication avec le matériel ne se fait que dans SOO et non pas dans la ME. Afin que la ME puisse tout de même interagir avec le matériel, il faut implémenter des interfaces virtuelles pour l'affichage de l'interface graphique et le support des périphériques d'entrée. Le développement se fera d'abord dans un environnement émulé (QEMU), puis il s'agira de l'adapter pour être déployé sur une Raspberry Pi 4.

Chapitre 2

Interface graphique

2.1 Choix de LittlevGL

Le choix de la librairie graphique s'est porté sur LittlevGL, librairie open-source écrite en C et sous licence MIT [5]. Elle a été conçue spécifiquement pour les systèmes embarqués et par conséquent ne requiert que peu de ressources. La librairie permet aussi la copie des pixels de l'interface graphique dans un framebuffer. Ce dernier étant une portion de la mémoire vive dans laquelle s'écrit une représentation binaire d'un ensemble de pixels afin de d'être affichés à l'écran.

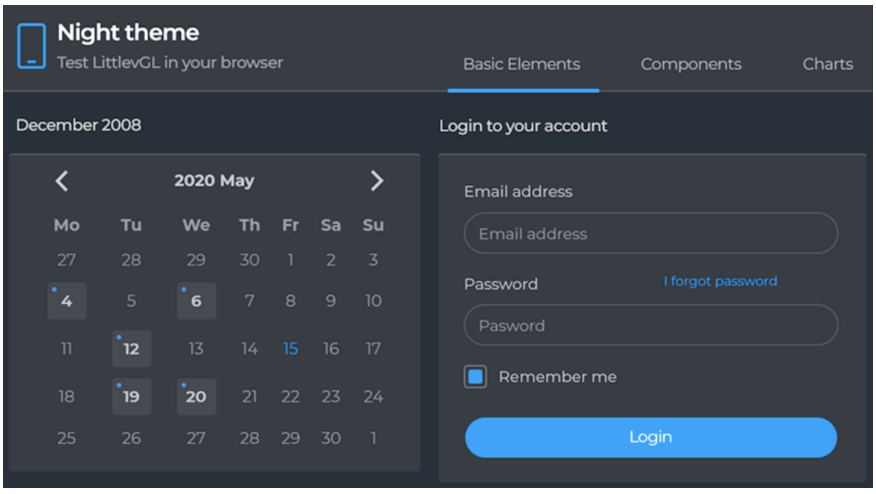


FIGURE 2.1 – Exemple d'interface graphique tel que présenté sur le site web de LittlevGL

Initialement, cette librairie ne devait être utilisée que dans un premier temps pour faciliter l'intégration d'une autre librairie plus complète (telles que Qt ou GTK). Cependant, au vu

des nombreuses fonctionnalités offertes par LittlevGL, de sa documentation très complète et d'une communauté active, il a été décidé de garder cette librairie jusqu'à la fin du travail de Bachelor.

Avec la sortie de la version 7 en mai 2020, le projet est renommé en LVGL (*Light and Versatile Graphic Library*). Cependant, l'appellation LittlevGL est gardée dans ce rapport car c'est la version 6.1.2 (février 2020) qui a été intégrée à SO3.

2.2 Intégration et configuration

La librairie est composée d'un répertoire nommé `lvgl` contenant son code source ainsi que plusieurs fichiers Makefile indiquant le chemin des sources devant être compilées. Il n'existe cependant pas de Makefile compilant la libraire.

Un fichier de configuration `lv_conf.h` est aussi fourni et doit être placé au même niveau que le répertoire `lvgl`. Il contient plusieurs paramètres tels que la résolution maximale que doit supporter la librairie, le nombre de bits par pixel, le niveau de log à afficher, etc.

La résolution maximale a été fixée à 1920×1080 ce qui est suffisant dans notre contexte de système embarqué. Quant à la résolution effective — qui est limitée par celle de l'écran utilisé — elle est communiquée à LittlevGL par l'application utilisateur. Un autre paramètre modifié est le nombre de bits par pixels, fixé à 32 — un octet pour chacune de ces valeurs : la transparence, le rouge, le vert et le bleu.

La librairie a été placée dans l'espace utilisateur de SO3 et non pas dans le noyau, car ce sont les applications utilisateur qui vont en faire usage. Elle a été placée dans le répertoire `so3/usr/lvgl`. L'arborescence est donc la suivante :

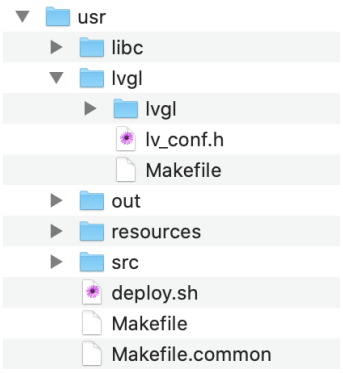


FIGURE 2.2 – Arborescence de `so3/usr/`

2.3 Compilation

Il a fallu écrire le fichier `lvgl/Makefile` afin que celui-ci produise une librairie statique pouvant être liée à l'exécutable de chacune des applications écrites dans `usr/src/`.

Le contenu de ce Makefile reste relativement simple, la cible `all` s'occupe de compiler tous les fichiers C — lesquelles sont indiqués par les fichiers Makefile (`*.mk`) susmentionnés — et de regrouper les fichiers objets résultants dans l'archive `liblvgl.a` avec la commande `ar`.

Il s'agissait ensuite de modifier le Makefile de `usr/` afin de lier la librairie au fichier objet de chaque application. Pour cela, une nouvelle cible créant la librairie a été écrite, puis celle-ci a été rajoutée comme dépendance de la cible créant l'exécutable (fichier ELF) de chacune des applications.

```
1 out/%.elf: src/%.o $(LIBC_DIR)/$(NLIB) $(LVGL_DIR)/$(LVGL_LIB)
2     $(LD) -o $@ libc/crt0.o libc/crt1.o $< \
3           -L$(LIBC_DIR) -lso3 \
4           -L$(LVGL_DIR) -llvgl $(LDFLAGS)
5
6 $(LVGL_DIR)/$(LVGL_LIB):
7     make -C lvgl all
```

Listing 2.1 – `usr/Makefile` : ajout de la cible compilant la librairie

Ainsi, une application utilisateur de SO3 peut désormais utiliser les fonctions définies par LittlevGL simplement en incluant le fichier `lvgl/lvgl.h`.

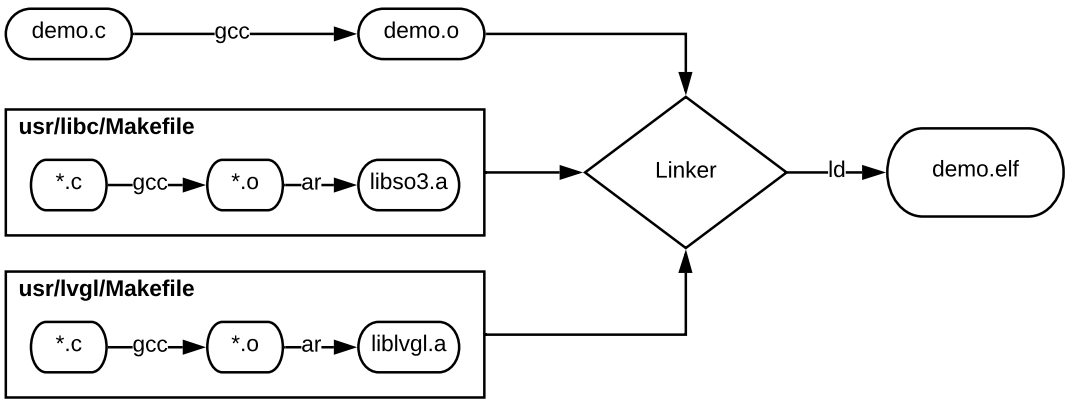


FIGURE 2.3 – Vue générale de la compilation d'une application SO3

2.4 Utilisation et fonctionnement

Toute application voulant utiliser LittlevGL doit appeler certaines fonctions et initialiser certaines structures de la librairie. Connaître ces différents éléments permet de mieux définir et comprendre les modifications qui devront être apportées à SO3. L'extrait suivant montre de manière simplifiée un exemple d'application LittlevGL (un exemple complet et fonctionnel a été écrit dans `usr/src/demo.c`).

```c
#include "lvgl/lvgl.h"

int main(int argc, char **argv)
{
    lv_init(); /* Internal LittlevGL initialization. */

    fs_init(); /* User-defined initializations. */
    fbdev_init();
    inputdev_init();

    create_ui();

    pthread_t thread; /* LittlevGL must know how time passes by. */
    pthread_create(&thread, NULL, tick_routine, NULL);

    while (1) { /* Run LittlevGL-defined tasks. */
        lv_task_handler();
        usleep(5000);
    }

    return 0;
}

void *tick_routine(void *args)
{
    while (1) { /* Tell LittlevGL that 5ms passed. */
        usleep(5000);
        lv_tick_inc(5);
    }
}
```

Listing 2.2 – Exemple d'application LittlevGL

LittlevGL initialise plusieurs de ses modules grâce à la fonction `lv_init`, son appel est donc obligatoire [10]. Les fonctions d'initialisation `*_init` qui suivent sont définies par le développeur et doivent initialiser certaines structures de LittlevGL. Ces structures seront vues plus en détails par la suite.

Deux autres autres composants sont essentiels, le premier est un thread qui indique à intervalle régulier le temps écoulé entre deux appels de la fonction `lv_tick_inc` [13] (fonction

définie par LittlevGL). Cela permet à la librairie de savoir quand exécuter certaines animations et tâches (e.g. récupération des coordonnées de la souris). Le deuxième est l'appel toutes les 5 millisecondes (valeur recommandée par LittlevGL) de `lv_task_handler`. Cette fonction s'occupe d'exécuter des tâches définies soit par la librairie (e.g. rafraîchissement de l'interface si changements) soit par le développeur (ce n'est pas le cas ici) [12].

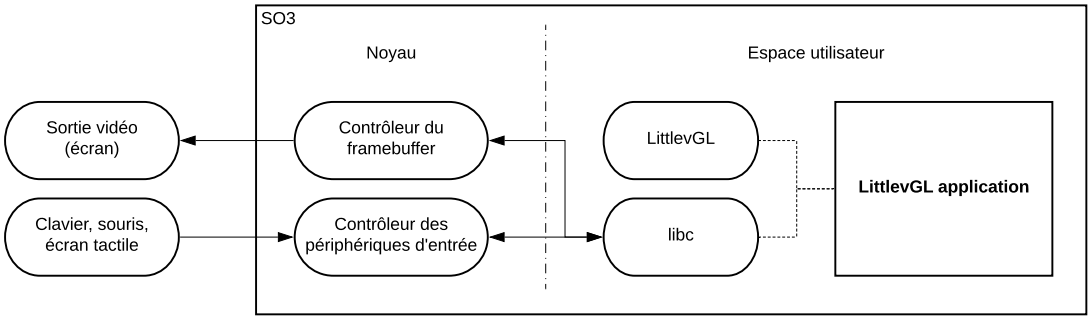


FIGURE 2.4 – Vue générale d'une application LittlevGL et de son interaction avec SO3

2.4.1 Affichage de l'interface

2.4.1.1 Description et utilisation du framebuffer

Afin d'afficher l'interface graphique générée par LittlevGL à l'écran, il faut pouvoir accéder et écrire dans le framebuffer depuis l'espace l'utilisateur de SO3.

Le framebuffer et un espace mémoire [23] dans lequel est écrit un ensemble de pixels. Il existe aussi un contrôleur de périphérique (circuit électronique) dédié au framebuffer et permettant de le configurer (par exemple pour décider de comment est représenté un pixel dans la mémoire — nombre de bits par couleur, ordre des couleurs, etc.).

Par exemple, avec 24 bits par pixel (*bpp* — 8 pour chaque couleur rouge, vert et bleu) il est possible de représenter plus de 16 millions de couleurs. Chaque pixel est alors représenté sur quatre octets, le premier est soit ignoré soit utilisé pour l'opacité, les suivants sont interprétés différemment selon la configuration du contrôleur en RGB ou BGR (inversion du rouge et du bleu).

Afin de communiquer avec ce contrôleur, il est nécessaire d'écrire un driver qui ne se situe pas dans l'espace utilisateur mais dans le noyau. Un driver est écrit pour un contrôleur spécifique et chaque système embarqué possède son propre ensemble de contrôleurs (à moins que des contrôleurs existants soit réutilisés). C'est donc avec le driver du framebuffer qu'il faut communiquer depuis l'espace utilisateur.

Dans le noyau Linux ainsi que dans les autres systèmes d'exploitation s'inspirant d'Unix, il existe des fichiers spéciaux [22] (ou fichiers de périphérique) qui n'existent pas sur le système de fichiers (e.g. FAT, NTFS) mais lesquels donnent accès à un périphérique. Par exemple, un framebuffer est représenté par un fichier nommé `/dev/fbX`, `X` étant un identifiant numérique. Cette technique est intéressante car elle permet aussi de gérer d'autres périphériques tels que ceux d'entrée comme le clavier et la souris, avec des noms de fichier comme `/dev/keyboardX` ou `/dev/mouseX`.

Bien que SO3 puisse déjà ouvrir des fichiers normaux, les fichiers spéciaux ne sont pas gérés et il a été décidé de les implémenter. L'implémentation est décrite dans la section 3.2.

Dans Linux, une fois le fichier du framebuffer ouvert et le descripteur de fichier récupérer (grâce à l'appel système `open`), une écriture dans ce fichier correspondra à une écriture dans le framebuffer. Cependant, utiliser l'appel système `write` de manière répétitive impactera les performances de l'application.

Une technique plus performante consiste à rendre accessible l'espace mémoire dédié au framebuffer dans l'espace utilisateur. Cela passe par l'appel système `mmap` et il n'est requis qu'une seule fois. Un pointeur sur l'espace mémoire est alors récupéré et le framebuffer peut être écrit. Cet appel système n'existe pas non plus dans SO3 et doit donc être implémenté.

Concrètement, voici le code nécessaire afin d'écrire un pixel dans le framebuffer. Ce système est aussi ce qui permettra de copier l'interface graphique de LittlevGL dans le framebuffer.

```
#include <fcntl.h>
#include <sys/mman.h>

int fd = open("/dev/fb0", O_WRONLY); /* Open the framebuffer. */
int *fbp = mmap(NULL, FB_SIZE, 0, 0, fd, 0); /* Map the memory. */
fbp[0] = 0x00ff0000; /* Write a red pixel in 24bpp RGB. */
```

Listing 2.3 – Exemple d'écriture dans un framebuffer

2.4.1.2 Driver d'affichage LittlevGL

Dans la nomenclature de la librairie, un *driver* correspond a une structure C permettant d'activer une fonctionnalité. À ces drivers sont attachées une ou plusieurs fonctions *callback* qui sont appelées par la librairie au moment propice.

Par exemple, au driver d'affichage [8] (*display driver*) il faut associer un buffer que LittlevGL utilisera pour redessiner l'interface. Un callback doit aussi être renseigné lequel copie ce buffer dans le framebuffer. Si ce buffer est plus petit que l'écran, l'interface sera redessinée en plusieurs fois. Il y a donc un compromis à faire entre utilisation de la mémoire et utilisation du CPU. À noter que LittlevGL ne va redessiner que les parties de l'interface qui

ont changées.

Il est aussi possible de fournir à LittlevGL un deuxième buffer. Dans ce cas, les opérations de rafraîchissement de l'interface et de copie dans le framebuffer peuvent être faites en parallèle. Il faut cependant que l'OS supporte l'accès direct à la mémoire (*Direct Memory Access*) ce qui n'est pas le cas de SO3, ce deuxième buffer n'a donc pas été utilisé dans notre cas.

Le code source suivant montre comment doit être initialisé le driver ainsi que le fonctionnement du callback qui utilise le pointeur sur le framebuffer récupéré précédemment.

```
static lv_color_t buf[LVGL_BUF_SIZE]; /* Buffer initialization. */
static lv_disp_buf_t disp_buf;
lv_disp_buf_init(&disp_buf, buf, NULL, LVGL_BUF_SIZE);

static lv_disp_drv_t disp_drv; /* Driver initialization. */
lv_disp_drv_init(&disp_drv);
disp_drv.hor_res = scr_hres;
disp_drv.ver_res = scr_vres;
disp_drv.buffer = &disp_buf;
disp_drv.flush_cb = my_fb_cb;
lv_disp_drv_register(&disp_drv);
```

Listing 2.4 – LittlevGL : initialisation du driver d'affichage

```
void my_fb_cb(lv_disp_drv_t *disp,
              const lv_area_t *area,
              lv_color_t *pixel)
{
    lv_coord_t y, line_width = lv_area_get_width(area);
    uint32_t line_size = line_width * sizeof(lv_color_t);

    for (y = area->y1; y <= area->y2; y++) { /* Copy line-by-line. */
        memcpy(&fbp[y * scr_hres + area->x1], pixel, line_size);
        pixel += line_width;
    }

    lv_disp_flush_ready(disp);
}
```

Listing 2.5 – Fonction callback de copie du buffer LittlevGL dans le framebuffer

2.4.2 Gestion des périphériques d'entrée

Un accès au clavier et à la souris peut être obtenu de la même manière que cela a été expliqué pour le framebuffer. Dans SO3, les drivers pour le clavier et la souris n'existent pas, ils doivent donc être implémentés.

Une fois le descripteur de fichier obtenu, une opération de lecture ou d'écriture n'aurait que peu de sens à moins de définir un protocole d'échange de données (e.g. récupération des coordonnées de la souris). Les OS inspirés d'Unix ont mis en place un appel système nommé `ioctl` [24] permettant d'exécuter des opérations définies par un driver.

Cet appel système existe déjà dans SO3, seules les opérations nécessaires doivent être implémentées. Pour le clavier, seule une opération pour récupérer la dernière touche pressée est requise. Pour la souris, deux opérations doivent être écrites. La première pour récupérer les coordonnées de la souris et la deuxième afin d'indiquer au driver la résolution de l'écran. Cette information est nécessaire afin que le driver puisse limiter les coordonnées de la souris, afin d'empêcher celle-ci de « sortir » de l'écran.

2.4.2.1 Driver d'entrée – Souris

L'initialisation du driver d'entrée LittlevGL [11] pour la souris et similaire à celle du framebuffer. La fonction callback exécutera l'opération pour récupérer les coordonnées de la souris et l'état de ses boutons. L'opération pour indiquer la résolution de l'écran doit être exécutée après l'obtention du descripteur de fichier de la souris.

```
int mfd = open("/dev/mouse0", 0);

struct display_res res = { .h = scr_hres, .v = scr_vres };
ioctl(mfd, IOCTL_MOUSE_SET_RES, &res);

lv_indev_drv_t mouse_drv; /* Initialisation du driver. */
lv_indev_drv_init(&mouse_drv);
mouse_drv.type = LV_INDEV_TYPE_POINTER;
mouse_drv.read_cb = my_mouse_cb;
```

Listing 2.6 – LittlevGL : initialisation du driver pour la souris

```
bool my_mouse_cb(lv_indev_drv_t *indev, lv_indev_data_t *data)
{
    static struct ps2_mouse mouse;
    ioctl(mfd, IOCTL_MOUSE_GET_STATE, &mouse);

    data->state = mouse->left ? LV_INDEV_STATE_PR : LV_INDEV_STATE_REL;
    data->point.x = mouse->x;
    data->point.y = mouse->y;

    return false;
}
```

Listing 2.7 – Fonction callback pour récupérer les coordonnées de la souris

2.4.2.2 Driver d'entrée – Clavier

L'initialisation du driver LittlevGL pour le clavier suis exactement le même principe. La fonction callback s'occupe par contre de récupérer la dernière touche appuyée du clavier grâce à un l'appel `ioctl`.

```
bool my_keyboard_cb(lv_indev_drv_t *indev, lv_indev_data_t *data)
{
    static struct ps2_key key;
    ioctl(kfd, IOCTL_KB_GET_KEY, &key);

    if (key.value != 0) {
        data->key = key.value;
        data->state = key.state & 1;
    }

    return false;
}
```

Listing 2.8 – Fonction callback pour récupérer la dernière touche appuyée du clavier

2.4.3 Accès au système de fichiers

LittlevGL fournit aussi un driver [9] pour accéder à des systèmes de fichiers de l'OS. Cela permet par exemple d'afficher des images dans l'interface graphique. Il faut initialiser un driver de ce type pour chaque système de fichiers devant être accéder par LittlevGL. De manière similaire à ce qui est fait sous Windows, une lettre est associée à chaque instance de driver permettant ainsi d'identifier les fichiers (e.g. `S:/mon-image.png`). SO3 ne possédant qu'un seul système de fichiers, seul un driver est nécessaire.

Les extraits suivants montrent le code nécessaire afin d'afficher une image dans l'interface graphique. Comme pour les drivers vus précédemment, plusieurs fonctions callback sont utilisées. Ici, elles servent à faire le lien entre LittlevGL et les fonctions C existantes de manipulation de fichier (e.g. `fopen`, `fread`, `fclose`). Dans SO3, ces fonctions sont fournies par musl [14], une librairie qui implémente la librairie C standard.

```
lv_fs_drv_t drv;
lv_fs_drv_init(&drv);

drv.letter = 'S';
drv.file_size = sizeof(FILE*);
drv.rddir_size = sizeof(FILE*);
drv.ready_cb = fs_ready_cb;     /* Drive is ready to use. */
drv.open_cb = fs_open_cb;       /* Callback to open a file. */
drv.close_cb = fs_close_cb;     /* Callback to close a file. */
drv.read_cb = fs_read_cb;       /* Callback to read a file. */
```

```
drv.seek_cb = fs_seek_cb;       /* Callback to set file cursor. */
drv.tell_cb = fs_tell_cb;       /* Callback to get file cursor. */
lv_fs_drv_register(&drv);
```

Listing 2.9 – LittlevGL : initialisation d'un driver de système de fichiers

```
lv_fs_res_t fs_open_cb(struct _lv_fs_drv_t *drv,
                       void *file_p,
                       const char *path,
                       lv_fs_mode_t mode)
{
    FILE *fp = fopen(path, (mode & LV_FS_MODE_WR) ? "w" : "r");
    if (!fp) {
        return LV_FS_RES_UNKNOWN;
    }

    *((FILE **)file_p) = fp;
    return LV_FS_RES_OK;
}

lv_fs_res_t fs_read_cb(struct _lv_fs_drv_t *drv,
                       void *file_p,
                       void *buf,
                       uint32_t btr, /* bytes to read */
                       uint32_t *br) /* bytes read */
{
    *br = fread(buf, sizeof(uint8_t), btr, *(FILE **)file_p);
    return LV_FS_RES_OK;
}
```

Listing 2.10 – Exemples de callbacks pour l'ouverture et la lecture de fichiers

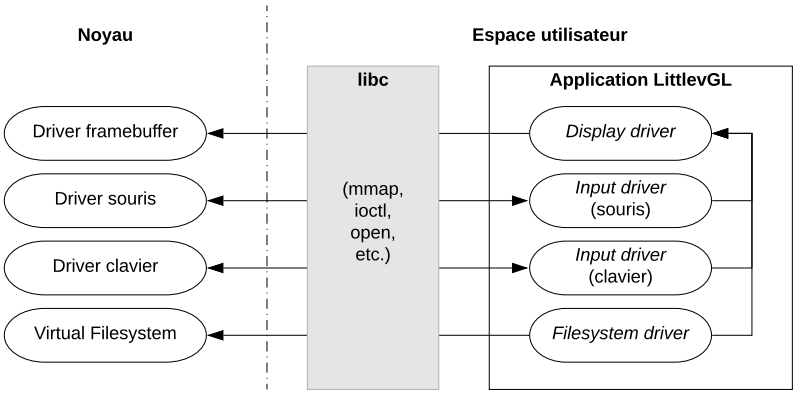


FIGURE 2.5 – Vue générale des drivers LittlevGL et de leurs interactions avec SO3

La figure 2.5 illustre les interactions des drivers LittlevGL vu précédemment avec le noyau SO3. Les drivers de SO3 ainsi que le *Virtual Filesystem* seront abordés dans le chapitre suivant.

2.4.4 Application de démonstration

Les extraits de code précédents font partie d'une application de démonstration écrite afin de démontrer les fonctionnalités de LittlevGL mais aussi, et surtout, de pouvoir s'assurer du bon fonctionnement des modifications qui seront apportées à SO3. Cette application se trouve dans le fichier `usr/src/demo.c`.

L'interface graphique de l'application (figures 2.6 et 2.7), composée de trois onglets, est basé sur du code de démonstration de LittlevGL disponible sur leur dépôt Github [7]. Le reste de l'application a été développé spécifiquement pour SO3.

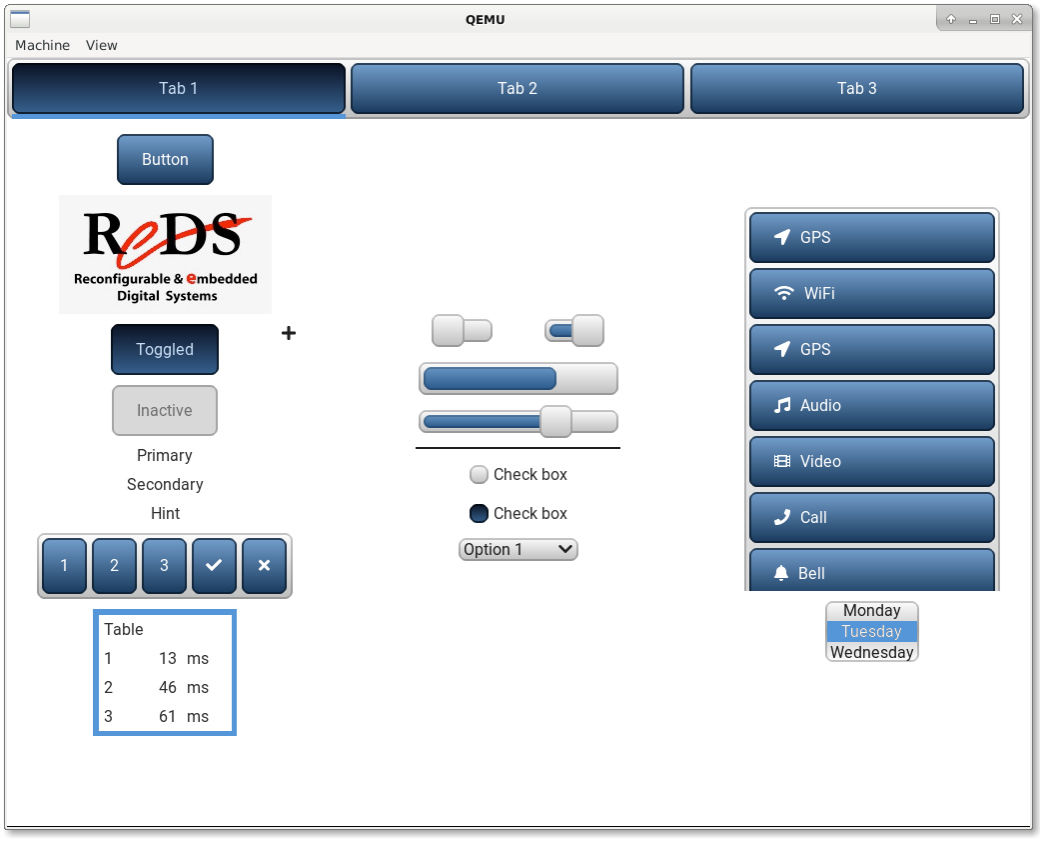


FIGURE 2.6 – Premier onglet de l'application de démonstration avec affichage d'une image

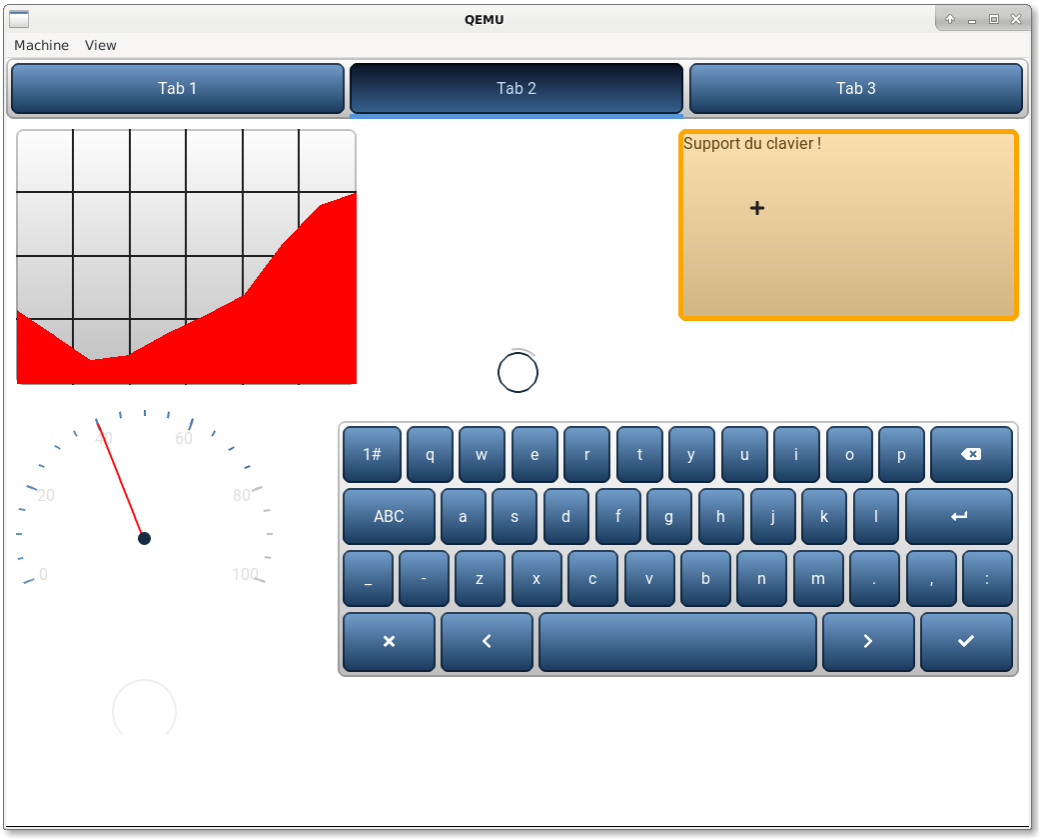


FIGURE 2.7 – Deuxième onglet de l'application de démonstration avec un champ texte

Chapitre 3

SO3

SO3 est un système d'exploitation entièrement écrit par le REDS à partir de zéro et dédié exclusivement aux processeurs ARM. Il n'est pas basé sur le noyau Linux et est un OS plus léger mais aussi plus facile à étudier et à modifier par des personnes débutant en développement de systèmes d'exploitation.

Le chapitre précédent a décrit le fonctionnement de LittlevGL mais a aussi introduit les modifications qui doivent être apportées à SO3 afin que cette librairie puisse fonctionner. Ces modifications seront détaillées dans ce chapitre.

3.1 Environnement de développement et d'exécution

3.1.1 QEMU

Afin de faciliter le développement de SO3, l'OS n'est pas déployé à chaque modification sur un système embarqué telle que la Raspberry Pi 4 mais est exécuté sur une plateforme émulée par le logiciel QEMU. Cela permet de tester les modifications apportées plus rapidement.

Actuellement, le REDS utilise QEMU pour émuler la plateforme ARM Versatile Express avec le processeur Cortex-A15. Une fois SO3 lancé sur cette plateforme, il est possible de communiquer avec via une interface UART. Cette interface permet d'envoyer et de recevoir des caractères entre l'OS utilisé pour le développement et SO3. Cependant, il est important de comprendre que cela ne nécessite du côté de SO3 ni un driver pour le clavier ni pour le framebuffer. Il faut un driver UART mais celui-ci existe déjà dans SO3.

SO3 n'ayant pas de support graphique, QEMU est lancé avec l'option `-nographic` qui désactive la sortie graphique. Il est nécessaire de retirer cette option afin que QEMU puisse afficher par la suite le contenu du framebuffer. L'option `-display gtk` peut aussi être utilisée mais `gtk` en est la valeur par défaut. Aussi, QEMU doit avoir été compilé avec l'option

`--enable-gtk` pour que l'option précédente fonctionne. Une fois ces manipulations faites et QEMU lancé, une nouvelle fenêtre affichera le message *Guest has not initialized the display (yet).*

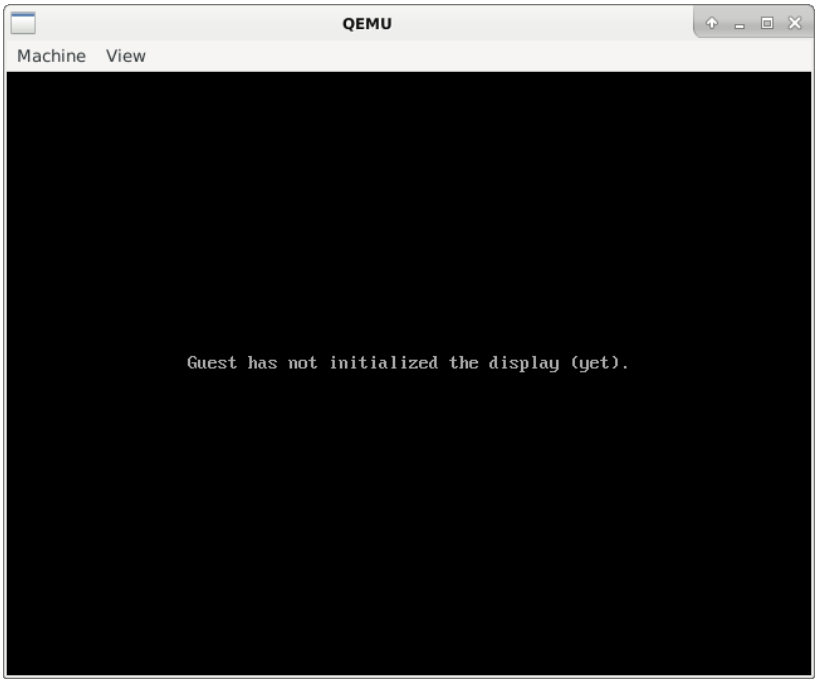


FIGURE 3.1 – Fenêtre QEMU pour l'affiche de l'interface graphique

3.1.2 ARM Versatile Express

La plateforme Versatile Express est composée de plusieurs contrôleurs de périphérique chacun ayant une fonction définie. Pour l'affichage de l'interface graphique, le contrôleur PL111 [2] (*PrimeCell ColorLCD Controller*) permet de gérer le framebuffer. Pour communiquer avec la souris et le clavier, il faut passer par le contrôleur PL050 [1] (*PrimeCell PS2 Keyboard/Mouse Interface*). Il s'agit donc d'écrire un driver pour chacun de ces contrôleurs afin que SO3 puisse les utiliser.

3.2 Fichiers spéciaux

Comme cela a été vu lors de l'utilisation de LittlevGL, afin de pouvoir communiquer avec le driver d'un contrôleur depuis l'espace utilisateur, il est nécessaire que le driver soit représenté par un fichier spécial. Ce fichier est dit spécial car il n'a pas de contenu et n'existe

pas dans le système de fichiers. Cependant, il est considéré comme un fichier car il est ouvert avec l'appel système `open` qui retourne un descripteur de fichier.

Pour le nom de ces fichiers, il a été décidé d'utiliser la convention suivante et proche de ce qui est fait sur les systèmes inspirés d'Unix : `/dev/<device-class><device-id>`. `device-class` représente le type de contrôleur (e.g. `fb` pour les framebuffers, `mouse` pour les souris) et `device-id` permet de numéroter les contrôleurs de même type. Ces valeurs sont définies dans `so3/include/device/device.h`.

3.2.1 Virtual Filesystem

Le point d'entrée depuis l'espace utilisateur est l'appel système `open` qui est implémenté dans la fonction `do_open` de `so3/fs/vfs.c`, qui est la couche VFS (*Virtual Filesystem*) de SO3. Le VFS gère la manipulation de fichiers et de répertoires indépendemment du système de fichiers sous-jacent. Actuellement, SO3 utilise FAT. Si cela devait changer, il n'y aurait besoin d'adapter seulement le lien entre le VFS et le nouveau système de fichiers.

Actuellement, l'appel système `open` considère tout fichier passé en paramètre comme existant sur le système de fichiers. Cette implémentation doit donc être modifiée afin que les fichiers commençant par `/dev/` soient considérés comme des fichiers spéciaux.

Lors de l'ouverture d'un fichier par `do_open`, une structure de type `fd` est créée. Celle-ci est ensuite associée au descripteur de fichier global grâce au tableau `open_fds`. Un descripteur de fichier local est aussi créé et fait référence à un descripteur global mais est propre à chaque processus. C'est ce descripteur local qui est retourné dans l'espace utilisateur.

Cette structure est importante car elle contient une structure de type `file_operations` qui contient les opérations sur fichier définies pour le fichier en question. Pour les fichiers présents sur le système de fichiers, c'est ce dernier qui définit les opérations possibles. Par contre, pour les fichiers spéciaux, ces opérations sont propres à chaque périphérique, elles doivent donc être définies dans le driver du périphérique.

Il existe une correspondance entre ces opérations et les appels système. Par exemple, l'appel système `open` exécutera l'opération `open` du fichier. De même pour `read`, `write`, `close` ainsi que `ioctl`, `mmap` et d'autres.

La figure 3.2 montre comment est utilisé le VFS lors de l'ouverture d'une fichier. Le VFS récupère les opérations sur fichier d'une source parmi plusieurs (*NTFS* et *Drivers* n'existe pas dans l'implémentation actuelle, ils sont mentionnés à titre d'exemple) puis les stocke dans la structure `fd` du fichier ouvert. Ces opérations peuvent alors être récupérées ultérieurement.

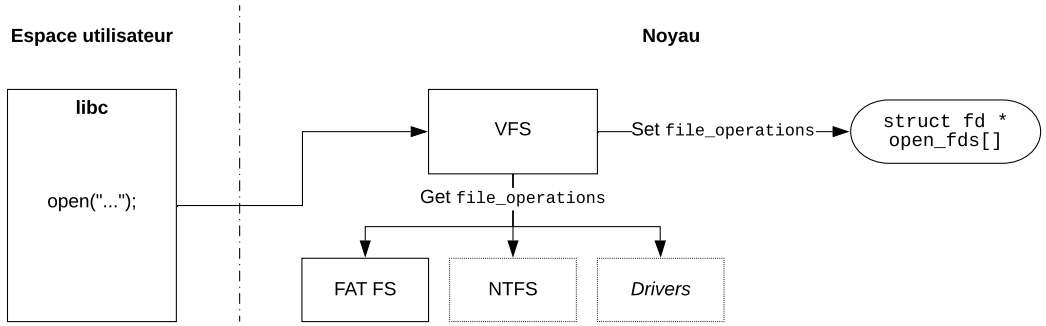


FIGURE 3.2 – Interactions avec le VFS lors de l'ouverture d'un fichier

3.2.2 Solution implémentée

Il faut donc, depuis la fonction `do_open`, être capable de récupérer les opérations sur fichier du driver correspondant au nom de fichier passé en paramètre. Pour cela, la structure `devclass` a été créée. Elle contient, entre autres, le type du contrôleur (e.g. `fb`) et les opérations sur fichier qui seront définies par le driver. Dans le fichier `so3/device/device.c`, une liste de structures `devclass` a été créée.

Pour rajouter un élément à cette liste, le driver utilise la fonction `devclass_register`. De son côté, le VFS utilise la fonction `devclass_get_fops` pour récupérer les opérations sur fichier du driver. Ces deux fonctions sont aussi définies dans `device.c`.

Le code source suivant illustre comment un driver déclare ses opérations. À noter qu'un driver n'est pas obligé de le faire, tous les drivers n'ont pas besoin d'être accessibles depuis l'espace utilisateur.

```
1  struct file_operations driver_fops = {
2      /* file operations to be defined */
3  };
4
5  struct devclass driver_cdev = {
6      .class = DEV_CLASS_, /* device class, defined in device.h */
7      .fops = &driver_fops,
8  };
9
10 int driver_init(dev_t *dev)
11 {
12     devclass_register(dev, &driver_cdev);
13 }
```

Listing 3.1 – Enregistrement des opérations sur fichier par un driver

L'extrait suivant montre une version simplifiée de `do_open` et de comment le VFS récupère les opérations sur fichier définie ci-dessus par le driver.

```
int do_open(const char *filename, int flags)
{
    int fd, gfd;
    struct file_operations *fops;

    /* Special file. */
    if (!strncmp(DEV_PREFIX, filename, DEV_PREFIX_LEN)) {

        /* Get fops defined by the driver. */
        fops = devclass_get_fops(filename + DEV_PREFIX_LEN, &type);
    }
    /* File on the filesystem. */
    else {
        /* Get fops of the FAT filesystem. */
        fops = registered_fs_ops[0];
    }

    /* Create struct fd entry in open_fds and stores fops. */
    fd = vfs_open(filename, fops, type);

    /* Global file descriptor, created in vfs_open. */
    gfd = current()->pcb->fd_array[fd];

    /* Call the open file operation, if defined. */
    if (fops->open)
        fops->open(gfd, filename);

    /* Return local file descriptor. */
    return fd;
}
```

Listing 3.2 – Récupération des opérations sur fichier dans l'appel système `open`

Ainsi, avec ce nouveau système mis en place, il devient possible de récupérer depuis l'espace utilisateur un descripteur de fichier lié à un périphérique. Avec ce descripteur, il est ensuite possible d'exécuter d'autres opérations comme `mmap` et `ioctl`, ce qui nous sera utile par la suite.

La figure 3.3 illustre l'implémentation des fichiers spéciaux dans SO3, en comparant notamment quand est-ce que le driver enregistre ses opérations sur fichier et quand est-ce qu'elles sont récupérées par le VFS.

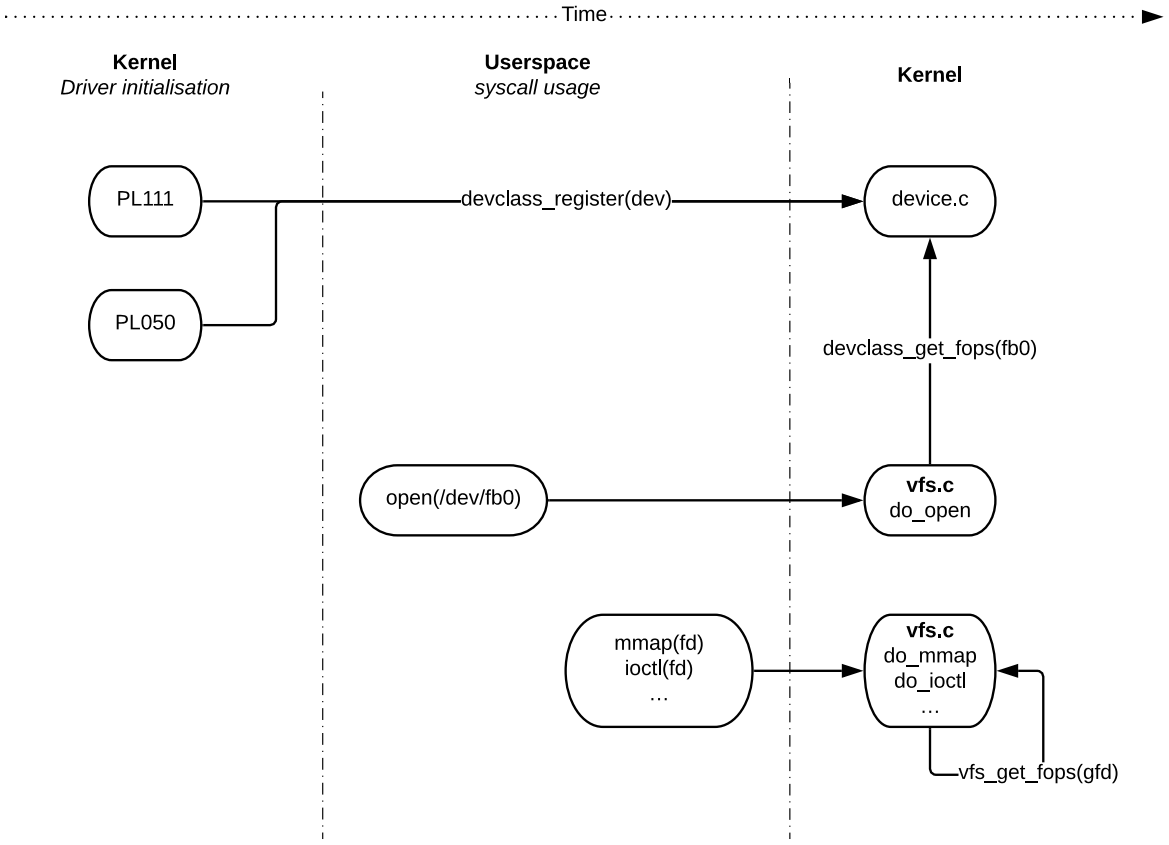


FIGURE 3.3 – Vue général de l'implémentation des fichiers spéciaux dans SO3

3.3 Driver du framebuffer

Afin de pouvoir écrire le driver du framebuffer, il est d'abord nécessaire d'identifier le contrôleur correspondant de la plateforme utilisée. Sur la plateforme ARM Versatile Express, ce contrôleur est le PL111 [4].

Le répertoire `fb` a été créé dans `so3/devices` et le driver y a été écrit dans le fichier `pl111.c`.

3.3.1 Description du PL111

Le PL111 possède un certain nombre de registres qui permettent de l'activer et de le configurer. La figure 3.4 extraite du document *PL111 Technical Reference Manual* [4] détaille une partie de ces registres.

Name	Base offset	Type	Reset value	Description
LCDTiming0	0x000	R/W	0x00000000	*Horizontal Axis Panel Control Register* on page 3-5
LCDTiming1	0x004	R/W	0x00000000	*Vertical Axis Panel Control Register* on page 3-6
LCDTiming2	0x008	R/W	0x00000000	*Clock and Signal Polarity Control Register* on page 3-8
LCDTiming3	0x00C	R/W	0x00000	*Line End Control Register* on page 3-10
LCDUPBASE	0x010	R/W	0x00000000	*Upper and Lower Panel Frame Base Address Registers* on page 3-10
LCDLPBASE	0x014	R/W	0x00000000	*Upper and Lower Panel Frame Base Address Registers* on page 3-10
LCDControl	0x018	R/W	0x0000	*LCD Control Register* on page 3-12

FIGURE 3.4 – Registres du PL111 tels que décrits dans la documentation ARM

3.3.1.1 Activation

La procédure de mise en route du contrôleur est décrite dans la section 1.1.8 de ce document. Il est indiqué qu'il faut activer (i.e. mettre à 1) deux bits (LcdEn et LcdPwr) du registre de contrôle (détaillé dans la figure 3.6). La modification de ces deux bits fera disparaître le message *Guest has not initialized the display (yet)* affiché par QEMU.

3.3.1.2 Position du framebuffer

Le PL111 peut gérer un ou deux panels en fonction de sa configuration (bit LcdDual du registre LCDControl). Dans notre cas, seul un panel est nécessaire, le bit LcdDual est donc mis à 0.

Le PL111 nous permet de définir les emplacements des deux framebuffers dans la mémoire vive. L'adresse du premier panel doit être mise dans le registre LCDUPBASE (***u**pper **p**anel*), celle du deuxième panel dans LCDLPBASE (*lower panel*).

Ce sont des adresses physiques qui doivent être fixées et elles doivent être alignées sur quatre octets (i.e. les deux derniers bits sont à 0). Puisque nous n'utilisons qu'un seul panel, LCDLPBASE est mis à 0. La plateforme Versatile Express dédie 32 Mo de mémoire vidéo (*Video SRAM* ou simple VRAM) située à l'adresse `0x18000000`. Cette adresse a été utilisée pour LCDUPBASE.

La figure 3.5 montre différents schémas permettant de trouver cette adresse. Les schémas sont extraits du document *Motherboard Express µATX Technical Reference Manual*.

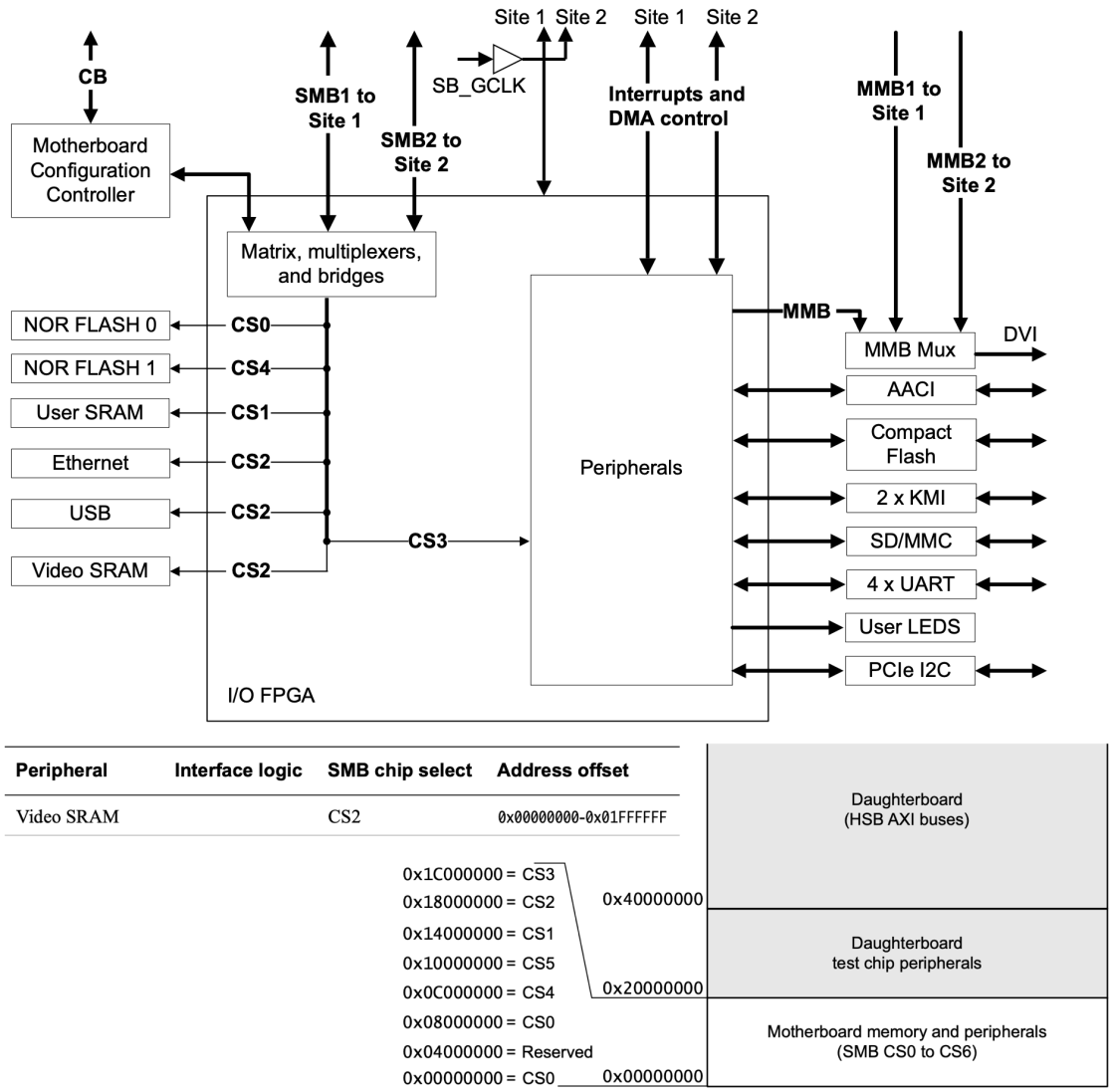


FIGURE 3.5 – L'adresse physique de la VRAM est `0x18000000`

3.3.1.3 Définition de la résolution

La résolution d'un écran LCD est définie par ses paramètres de *timing*. Il y a quatre registres de timing numérotés de 0 à 3. Les deux premiers sont les plus importants et permettent de configurer la résolution de l'écran.

Dans le premier de ces registres, six bits code le nombre de pixels par ligne (*pixels-per-line*, PPL). Cette valeur est définie par la formule $H/16-1$, H étant la résolution horizontale voulue. Dans le deuxième registre, dix bits code le nombre de lignes par panel (*lines-per-panel*, LPP). Elle est définie comme étant la résolution verticale moins 1.

Ces deux registres possèdent d'autres valeurs qui doivent être renseignées. Des exemples de valeurs pour les différentes propriétés de ces deux premiers registres peuvent être trouvées dans divers fichiers DTS (*Device Tree Source*) du noyau Linux.

Les deux derniers registres permettent de configurer différentes propriétés relatives aux signaux d'horloges.

3.3.1.4 Registre de contrôle

Le registre de contrôle est un des plus importants car il permet d'initialiser le contrôleur comme expliquer dans la section 3.3.1.1. Il permet aussi de définir, entre autres, le nombre de bits par pixel (*bits per pixel*, bpp), le type d'écran (TFT ou STN), le nombre de panels (1 ou 2) et si l'affichage du rouge et du bleu doit être inversé ou pas (BGR ou RGB).

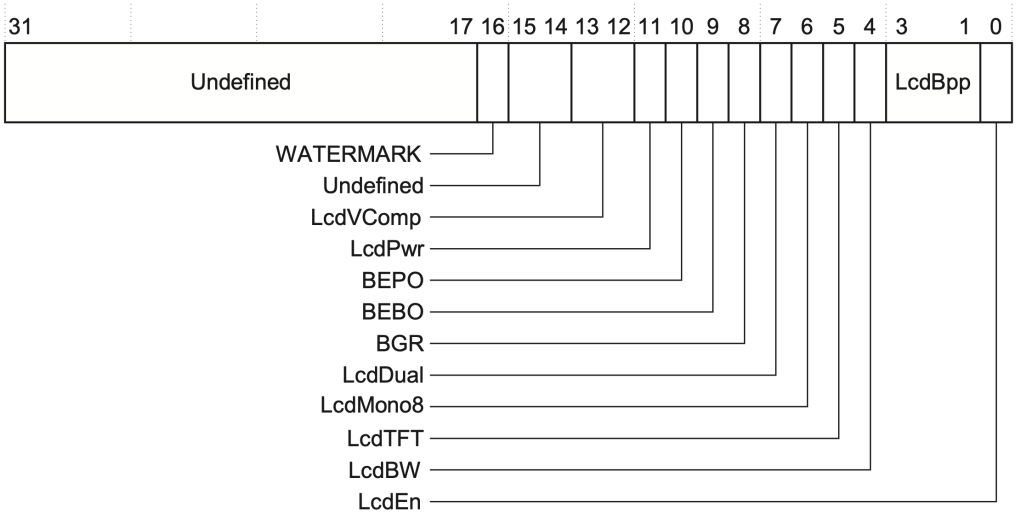


FIGURE 3.6 – Détail des bits du registre LCDControl [4]

3.3.1.5 Écriture des registres

L'écriture des valeurs susmentionnées dans les registres correspondants se fait dans la fonction d'initialisation `pl111_init` du driver. Celle-ci est enregistrée auprès du noyau grâce à la macro `REGISTER_DRIVER_POSTCORE`. Dans SO3, il existe des drivers CORE et POST-CORE, la distinction est faite pour permettre au noyau d'initialiser les drivers CORE en premier. La modification des registres se fait grâce à la fonction `iowrite32`.

3.3.2 Mapping de la mémoire

Dans l'application utilisateur utilisant LittlevGL, après avoir obtenu un descripteur de fichier sur le framebuffer, l'appel système `mmap` est utilisé afin de mapper une partie de la mémoire du processus vers celle du framebuffer. L'emplacement de la mémoire du processus est communiqué par l'appel `mmap` de la libc, donc depuis l'espace utilisateur.

L'implémentation de `mmap` dans le noyau a été rajoutée dans le VFS (fonction `do_mmap`). Son implémentation est simple. D'abord, le nombre de page devant être mappées est calculé en divisant la taille de la zone mémoire a être mappée (passée en paramètre) par la taille d'une page (constante du noyau). Ensuite, en utilisant le descripteur de fichier local passé en paramètre, le descripteur global est récupéré et à partir de celui-ci, les opérations sur fichier que le driver a définit. `mmap` étant une de ces opérations, elle est exécutée.

Ainsi, la logique centrale de l'appel système `mmap` est définie par le driver, et peut donc être différente pour chaque périphérique. Cette séparation est très similaire à ce qui est fait par les OS inspirés d'Unix.

Dans le driver du PL111, l'opération `mmap` est rajoutée. L'opération dispose de l'adresse virtuelle du processus à partir de laquelle doit être fait le mapping. Cette adresse est décidée par l'espace utilisateur. L'opération dispose aussi du nombre de pages à être mappées. Un mapping est effectué entre l'adresse virtuelle et l'adresse physique de la VRAM grâce à la fonction `create_mapping` définie dans `mmu.c`.

```
1  void *fb_mmap(int fd, uint32_t virt_addr, uint32_t page_count)
2  {
3      uint32_t i;
4
5      for (i = 0; i < page_count; i++)
6          create_mapping(current()->pcb->pgtable,
7                         virt_addr + i * PAGE_SIZE,
8                         LCDUPBASE + i * PAGE_SIZE,
9                         PAGE_SIZE, false);
10
11     return (void *) virt_addr;
12 }
13
```

```
14 struct file_operations pl111_fops = {
15     .mmap = fb_mmap
16 };
```

Listing 3.3 – Implémentation de `mmap` par le PL111

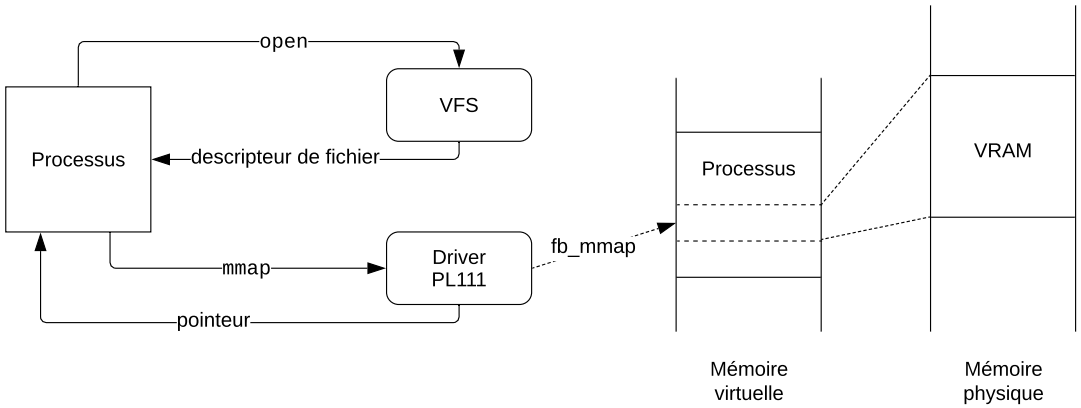


FIGURE 3.7 – Vue générale du mapping de la mémoire du framebuffer

3.3.3 Propriétés du framebuffer

Afin que l'espace utilisateur puisse correctement appeler l'appel système `mmap`, la taille de la zone mémoire correspondant au framebuffer doit être connue. Cette taille dépend de la résolution de l'écran (définie par le driver comme vu précédemment) et du nombre de bits par pixel (aussi défini par le driver).

La résolution de l'écran doit aussi être connue par l'application utilisateur pour être indiquée au driver d'affichage de LittlevGL ainsi qu'au driver de la souris afin que les coordonnées de celle-ci soient limitées par la taille de l'écran.

Afin de pouvoir récupérer ces propriétés, l'appel système `ioctl` a été utilisé. Cet appel existe déjà et n'a pas requis de modifications. Il a fallu cependant implémenter les opérations voulues dans le driver du PL111, à la manière de ce qui a été fait pour `mmap`.

`ioctl` prend en argument l'identifiant de la commande devant être exécutée ainsi qu'un entier qui est utilisé pour récupérer d'autres paramètres ou renvoyer une valeur de retour (il faut alors l'utiliser comme un pointeur). L'identifiant est définit par le driver mais il doit être constant, afin qu'il puisse aussi être connu de l'espace utilisateur.

Le code suivante montre l'implémentation de `ioctl` dans le driver.

```
int fb_ioctl(int fd, unsigned long cmd, unsigned long args)
{
    switch (cmd) {

    case IOCTL_HRES: /* Horizontal resolution. */
        *((uint32_t *) args) = HRES;
        return 0;

    case IOCTL_VRES: /* Vertical resolution. */
        *((uint32_t *) args) = VRES;
        return 0;

    case IOCTL_SIZE: /* Framebuffer size. */
        *((uint32_t *) args) = HRES * VRES * 4; /* assume 24bpp */
        return 0;

    default: /* Unknown command. */
        return -1;
    }
}
```

Listing 3.4 – Implémentation de l'opération `ioctl` par le PL111

3.4 Drivers du clavier et de la souris

À nouveau, l'identification du contrôleur permettant de communiquer avec le clavier et la souris est essentiel. Sur la plateforme ARM Versatile Express, ce contrôleur est le PL050 [3] et il y est présent en deux exemplaires. Le premier permet de communiquer avec le clavier (nommé KMI0) et le deuxième avec la souris (KMI1).

PS2 est le protocole utilisé pour la communication entre le driver et les périphériques d'entrée que sont le clavier et la souris.

Le répertoire `input` a été créé dans `so3/devices`. Un driver pour le clavier et un autre pour la souris ont été écrits et sont répartis dans plusieurs fichiers :

- `pl050.c` : code commun utilisé autant par la souris que par le clavier.
- `kmi0.c` : driver pour le clavier.
- `kmi1.c` : driver pour la souris.
- `ps2.c` : code utilitaire pour faciliter la communication avec la souris et le clavier.

3.4.1 Description du PL050

3.4.1.1 Activation

L'initialisation du contrôleur passe par deux registres : KMICLKDIV (*clock divisor register*) et KMICR (*control register*).

Indiqué dans la documentation ARM [3], l'horloge de référence doit générer un signal de 8 MHz. Il faut alors spécifier une valeur dans le registre KMICLKDIV qui sera utilisée pour diviser l'horloge de référence afin d'obtenir ces 8 MHz. La valeur 2 a été reprise du driver Linux.

Il faut ensuite modifier le registre de contrôle (KMICR) afin d'activer le contrôleur (bit KmiEn) et les interruptions émises lorsque des données sont reçues du périphérique (bit KMIRXINTREn). Les interruptions émises lorsque des données peuvent être envoyées au périphérique ne sont pas nécessaires et peuvent donc être désactivées (bit KMITXINTREn).

Il est aussi requis d'enregistrer auprès du contrôleur d'interruptions de SO3 la routine de service qui sera exécutée à chaque fois que des données envoyées par le périphérique peuvent être lues par le driver. Ceci est fait grâce à la fonction `irq_bind` définie dans `irq.c`.

L'initialisation est faite dans la fonction `pl050_init` du fichier `pl050.c`.

```
1  int pl050_init(dev_t *dev, dev_t *dev_copy, struct devclass *cdev,
2                 irq_return_t (*isr)(int, void *))
3  {
4      /* Set the clock divisor (arbitrary value). */
5      iowrite8(dev->base + KMI_CLKDIV, 2);
6
7      /* Enable transmitter and receiver interrupt. */
8      iowrite8(dev->base+KMI_CR, KMICR_EN | KMICR_RXINTREN | KMICR_TXINTREN);
9
10     /* Bind the ISR to the interrupt controller. */
11     irq_bind(dev->irq, isr, NULL, NULL);
12
13     /* Register the input device so it can be accessed from user space. */
14     devclass_register(dev, cdev);
15
16     return 0;
17 }
```

Listing 3.5 – Initialisation commune du driver du PL050

3.4.1.2 Envoi de données

Lorsque le driver souhaite envoyer des données au périphérique, il doit les écrire dans le registre KMIDATA (8 bits). Avant cela, le driver doit vérifier que ce registre soit vide, ceci

Address	Type	Width	Reset value	Name	Description
KMI Base + 0x00	Read/ write	6	0x00	KMICR	Control register.
KMI Base + 0x04	Read	7	0x43	KMISTAT	Status register.
KMI Base + 0x08	Read/ write	8/ 8	0x00	KMIDATA	Received data (read)/ Data to be transmitted (write).
KMI Base + 0x0c	Read/ write	4	0x00	KMICLKDIV	Clock divisor register.
KMI Base + 0x10	Read	2	0x00	KMIIR	Interrupt status register.

FIGURE 3.8 – Registres du PL050 tels que décrits par la documentation ARM [3]

est fait en s'assurant que le bit TXEMPTY du registre KMISTAT (*status register*) est à 1. Cela a été codé dans la fonction `pl050_write` du fichier `pl050.c`.

```
1  void pl050_write(dev_t *dev, uint8_t data)
2  {
3      /* Check if we can actually write in the transmit registry. */
4      uint8_t status = ioread8(dev->base + KMI_STAT);
5      if (0 == (status & KMISTAT_TXEMPTY)) {
6          printk("%s: write timeout.\n", __func__);
7          return;
8      }
9
10     iowrite8(dev->base + KMI_DATA, data);
11 }
```

Listing 3.6 – Écriture du registre KMIDATA du PL050

3.4.1.3 Réception de données

Lorsque le périphérique écrit des données dans le registre KMIDATA, la routine de service du driver est exécutée puisqu'une interruption est levée. Dans cette routine, il faut lire le contenu du registre KMIDATA tant que le bit KMIRXINTR du registre KMIIR (*interrupt status register*) est à 1.

Cela permet de regrouper les octets lus en un paquet PS2, lequel doit être analysé différemment selon qu'il provienne de la souris ou du clavier. La routine de service du clavier est la fonction `pl050_int_keyboard` du fichier `kmi0.c` et celle de la souris est la fonction `pl050_int_mouse` du fichier `kmi1.c`.

```
irq_return_t pl050_int_keyboard(int irq, void *dummy)
{
    /* Read the interrupt status register. */
    uint8_t status, packet[2], i, tmp;
    status = ioread8(pl050_keyboard.base + KMI_IR);

    /* As long as a receiver interrupt has been asserted */
    i = 0;
    while (status & KMIIR_RXINTR) {
        tmp = ioread8(pl050_keyboard.base + KMI_DATA);
        if (i < 2) /* Remember only the first 2 bytes. */
            packet[i++] = tmp;

        /* Update status. */
        status = ioread8(pl050_keyboard.base + KMI_IR);
    }

    get_kb_key(packet, i, &last_key); /* defined in ps2.c */
    return IRQ_COMPLETED;
}
```

Listing 3.7 – Récupération d'un paquet PS2 envoyé par le clavier

3.4.1.4 Traitement des paquets

Souris À chaque mouvement de la souris, celle-ci envoie une série de paquets contenant, entre autres, la différence de position de la souris (abscisse et ordonnée) par rapport au paquet précédent et l'état des boutons de la souris (s'ils ont été appuyés ou non). À partir de ces données, le driver calcule et garde en mémoire les coordonnées absolues de la souris ainsi que l'état de ses boutons. Ces données sont stockées dans une structure de type `ps2_mouse`. La fonction remplissant ces données est `get_mouse_state` du fichier `ps2.c`.

```
void get_mouse_state(uint8_t *packet, struct ps2_mouse *state, uint16_t
    max_x, uint16_t max_y)
{
    if (!state->left)
        state->left = packet[PS2_STATE] & BL;

    if (!state->right)
        state->right = packet[PS2_STATE] & BR;

    if (!state->middle)
        state->middle = packet[PS2_STATE] & BM;

    /* Retrieve dx and dy values to compute the mouse coordinates. */
    state->x += GET_DX(packet[PS2_STATE], packet[PS2_X]);
    state->y -= GET_DY(packet[PS2_STATE], packet[PS2_Y]);

```

```
16      state->x = CLAMP(state->x, 0, max_x);
17      state->y = CLAMP(state->y, 0, max_y);
18  }
```

Listing 3.8 – Analyse d'un paquet PS2 envoyé par la souris

Clavier Le traitement des paquets provenant du clavier est bien plus complexe [15]. En effet, il faut implémenter un mécanisme qui prenne en compte :

- les différents types de touches (certaines envoient un paquet de 2 octets, d'autres de 3 octets),
- l'état de la touche concernée (appuyée ou relâchée),
- la présence ou non de touches modifiant le comportement d'une autre touche ; tout en sachant que ces touches (e.g. Shift, Ctrl, Alt) peuvent être combinées et qu'elles sont chacune envoyées dans leur propre paquet,
- les différents types de comportements lorsqu'une touche est appuyée (A doit afficher un « a », Shift + A un « A », alors que Ctrl + A devrait pouvoir exécuter une action définie par l'espace utilisateur).

Et tout ceci sans prendre en compte la langue du clavier qui pourrait être définie et modifiée par l'utilisateur. Ainsi, pour simplifier l'écriture de ce driver, seules les fonctionnalités essentielles suivantes ont été implémentées :

- lettres minuscules,
- lettres majuscules (avec la touche Shift),
- quelques touches de contrôles (e.g. flèches, retour arrière, supprimer, tabulation).

Le driver traduit donc les *scan codes* (code d'identification d'une touche) présents dans les paquets reçus en un caractère UTF-8 et garde en mémoire le dernier code calculé. Ce code est stocké dans une structure de type `ps2_key`. Le traitement des paquets est fait dans la fonction `get_kb_state` du fichier `ps2.c`.

Afin de savoir comment traduire un scan code en un caractère UTF-8, plusieurs tableaux de traduction ont été codés dans `ps2.c`. Les éléments sont définis de la manière suivante : `tableau[scan code] = caractère UTF-8`. L'inconvénient de cette méthode est que tous les éléments des tableaux ne sont pas utilisés, il y a donc un gaspillage de mémoire.

```
1  void get_kb_key(uint8_t *packet, uint8_t len, struct ps2_key *key)
2  {
3      uint8_t i = 0;
4
5      /* If the first byte is KEY_EXT then we will use the extended scan codes
        array. */
```

```
6       key->state &= ~KEY_ST_EXTENDED;
7       if (packet[i] == KEY_EXT) {
8           key->state |= KEY_ST_EXTENDED;
9           i++;
10      }
11
12      /* Change the pressed/released state bit. */
13      key->state |= KEY_ST_PRESSED;
14      if (packet[i] == KEY_REL) {
15          key->state &= ~KEY_ST_PRESSED;
16          i++;
17      }
18
19      /* Change the shift modifier bit (left or right shift key). */
20      if (packet[i] == KEY_LSH || packet[i] == KEY_RSH) {
21          key->value = 0;
22          key->state ^= KEY_ST_SHIFT;
23      }
24      else {
25          /* Convert the scan code to the wanted value. */
26          if (key->state & KEY_ST_SHIFT)
27              key->value = scs[packet[i]];
28          else if (key->state & KEY_ST_EXTENDED)
29              key->value = sce[packet[i]];
30          else
31              key->value = sc[packet[i]];
32      }
33  }
```

Listing 3.9 – Analyse d'un paquet PS2 envoyé par le clavier

3.4.2 Coordonnées et touches appuyées

Pour récupérer la position de la souris et les touches appuyées du clavier depuis l'espace utilisateur, il faut passer par l'appel système `ioctl` qui permet d'exécuter une opération définie par le driver. Au préalable, il est nécessaire d'obtenir un descripteur de fichier pour le driver correspondant, de la même manière que pour le framebuffer.

Les opérations `ioctl` disponibles sont définies dans les fichiers `kmi0.c` et `kmi1.c` et sont les suivantes.

- `SET_SIZE` : permet d'indiquer au driver la résolution de l'écran, ainsi il peut poser une limite maximale aux coordonnées de la souris (afin que celle-ci ne sorte pas de l'écran). C'est une structure de type `display_res` qu'il faut passer en paramètre à l'opération.
- `GET_STATE` : permet de récupérer les coordonnées de la position actuelle de la souris sous la forme d'une structure de type `ps2_mouse`.

- `GET_KEY` : permet de récupérer la valeur de la dernière touche appuyée du clavier. La structure retournée est de type `ps2_key`.

Chapitre 4

SOO

4.1 Introduction

Dans la première partie de ce travail de Bachelor, le support graphique a été ajouté à SO3 lorsque celui-ci est exécuté de manière autonome, c'est-à-dire qu'il n'est pas virtualisé et donc qu'il possède un accès direct au matériel de la plateforme sur laquelle il tourne.

Dans cette deuxième partie, il s'agit d'intégrer les changements apportés à SO3 dans une ME qui sera virtualisée par la technologie SOO. Le travail sera effectué sur la ME nommée SOO.refso3 qui est basée, comme son nom l'indique, sur SO3.

Pour comprendre les changements qui devront être effectués, il est important de bien saisir les principaux composants de la technologie SOO et comment ils interagissent. L'hyperviseur AVZ (*Agency Virtualizer*), développé par le REDS, se lance en premier. Cet hyperviseur est de type 1 car c'est lui qui initie la virtualisation des systèmes d'exploitation. Le premier OS virtualisé est l'agence, un noyau Linux augmenté de plusieurs modifications apportées par le REDS qui permettent, entre autres, la communication avec les différentes ME qui sont virtualisées.

Autant l'agence que les différentes ME sont considérées comme des domaines. L'agence est le domaine 0 et est le seul domaine à avoir accès au matériel de la plateforme. Ainsi, dans une ME, il n'y a plus de notion de matériel. Pour ne pas avoir à modifier l'espace utilisateur de la ME, il va falloir créer des drivers virtuels qui auront pour but d'imiter le comportement des drivers de périphériques comme, par exemple, celui du PL111 ou du PL050.

Intercommunication Agence–ME Cependant, ces drivers virtuels auront quand même besoin de communiquer avec le matériel, que se soit pour écrire dans le framebuffer ou pour récupérer les événements des périphériques d'entrée. Pour cela, pour chacun des périphériques, un driver backend doit être créé dans l'agence et un driver frontend doit être créé

dans la ME. Le driver backend, ou simplement backend, a accès au matériel puisqu'étant dans l'agence. À noter qu'en général le backend ne communique pas directement avec le matériel mais profite des drivers Linux existants. Le frontend, présent dans la ME, communique avec le driver virtuel afin de simuler l'existence du périphérique.

Évidemment, la communication entre le backend et le frontend est nécessaire. Celle-ci est possible grâce à plusieurs mécanismes aussi implémentés par le REDS. Il y a, par exemple, le VBStore qui permet d'écrire et de lire — depuis l'agence ou la ME — une valeur à un chemin donné. Il existe aussi un anneau partagé, qui permet l'échange de requêtes (initiées par le frontend) et de réponses (initiées par le backend).

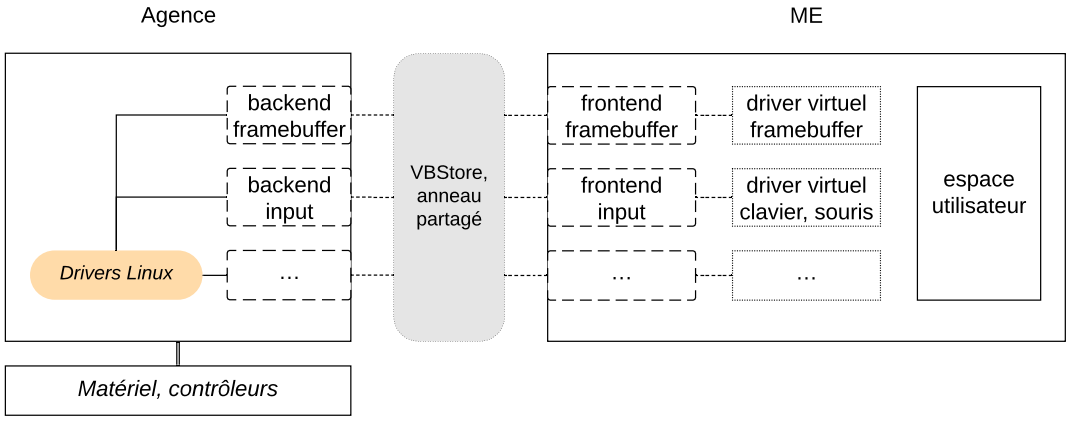


FIGURE 4.1 – Interactions entre les drivers Linux, ceux du backend et frontend, et les drivers virtuels

4.2 Gestion de plusieurs ME

Il faut noter que plusieurs ME peuvent être virtualisées en même temps. Il y aura donc plusieurs frontends pour le même périphérique. Par contre, il n'y aura toujours qu'un seul backend par périphérique. Ainsi, dans le cas ou plusieurs ME sont virtualisées, un backend communique avec plusieurs frontends.

Cela implique de savoir quel framebuffer de quel ME doit être affiché à l'écran à un moment donné. Il faut aussi savoir vers quelle ME doivent être redirigées les données des périphériques d'entrée. Lorsque l'agence a démarré et que l'accès à la ligne de commande Linux est donné, il est possible de naviguer parmi les domaines en tapant deux fois sur Ctrl + A.

Il a été décidé d'utiliser ce système afin de savoir quel domaine est actif à un instant donné et quand est-ce qu'on passe d'un domaine à un autre. Cela permet de décidé quel domaine doit voir son framebuffer affiché à l'écran et quel domaine doit recevoir les données des périphériques d'entrées.

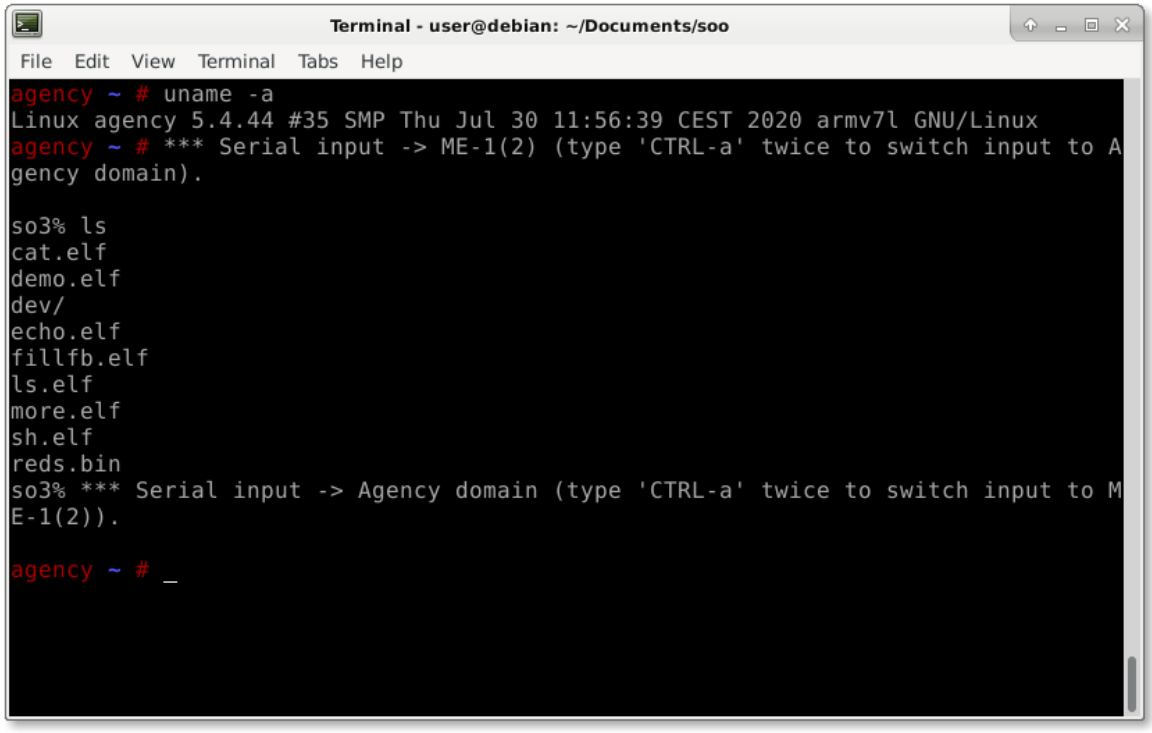


FIGURE 4.2 – Exemple d'utilisation du double Ctrl + A

C'est donc dans le fichier `linux/soo/kernel/console/console.c` que des modifications ont été apportées. Initialement, il n'était possible de naviguer qu'entre l'agence et la première ME. Une fonction plus générique nommée `get_next` a été créée qui retourne l'identifiant du prochain domaine. Pour cela, il a été nécessaire d'utiliser la fonction `get_ME_desc` afin de détecté si une ME est démarrée ou non.

Ensuite, dans la fonction `avz_switch_console`, l'identifiant du domaine actif est communiqué au backend du framebuffer et à celui des périphériques d'entrée (qui seront détaillés plus loin). Ainsi, ces backends sont au courant du domaine actif et peuvent donc agir en conséquence.

4.3 Gestion des framebuffers

4.3.1 Création du backend et frontend

Le backend a été crée dans `linux/soo/drivers/vfbback/vfb.c` et le frontend dans `SOO.refso3/so3/soo/drivers/vfbfront/vfb.c`. Le nom vfb correspond à *virtual framebuffer* et respecte la convention actuelle des autres backends et frontends (e.g. vuart, vdummy).

Afin que le backend et le frontend soient correctement compilés, un Makefile contenant le nom du fichier objet devant être obtenu a été créé dans les répertoires `vfbback` et `vfbfront`. Le nom du fichier étant `vfb.c`, par convention le nom du fichier objet sera `vfb.o`.

```
/* vfbback/Makefile */
obj-$(CONFIG_VFB_BACKEND) := vfb.o

/* vfbfront/Makefile */
obj-$(CONFIG_VFB_FRONTEND) := vfb.o
```

Listing 4.1 – Makefile du backend et frontend vfb

La variable `CONFIG_VFB_*` fait partie du système de configuration Kconfig utilisé par le noyau Linux et repris par SO3. Il permet d'activer ou de désactiver des composants du noyau (e.g. drivers). Désactiver implique que le composant n'est pas compilé et donc qu'un binaire plus petit est obtenu.

Il est alors possible de rendre la compilation du vfb optionnelle ce qui permettrait de continuer à utiliser des ME sans interface graphique. Il faut pour cela que les deux variables soient déclarées. Celle du backend doit l'être dans `soo/drivers/Kconfig` et celle du frontend dans `so3/soo/drivers/Kconfig`. L'extrait suivant illustre la déclaration de la variable du backend.

```
config VFB_BACKEND
    bool "vfb framebuffer backend driver"
    default N
    help
      This driver enabled to deal with a virtualized interface for a
    framebuffer
      in the Mobile Entity.
```

Listing 4.2 – Déclaration de la variable Kconfig pour le backend vfb

La configuration du noyau (Linux ou SO3) se fait avec la commande `make menuconfig` exécutée dans le répertoire contenant le Makefile principal. Cette commande fournit une interface pour activer ou désactiver les composants du noyau.

Puisque le backend et frontend sont considérés comme des drivers par le noyau, il est nécessaire que les fichiers DTS décrivant le matériel de la plateforme référencent ces deux

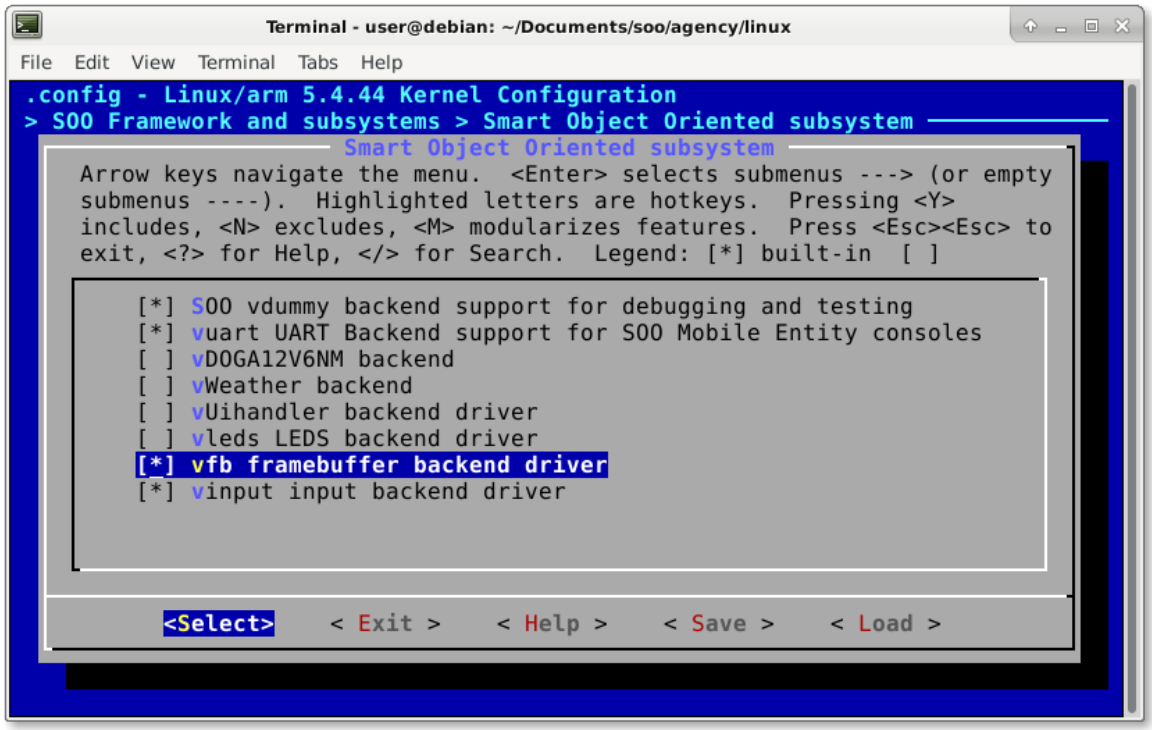


FIGURE 4.3 – Activation du vfb dans le menu Kconfig de l'agence

composants. L'extrait suivant montre les nœuds à ajouter. Le backend est rajouter dans `vexpress-tz.dts` (Linux) et le frontend dans `so3virt.dts` (SO3).

```
agency {
    backends {
        vfb {
            compatible = "vfb,backend";
            status = "ok";
        };
    }
}

ME {
    frontends {
        vfb {
            compatible = "vfb,frontend";
            status = "ok";
        };
    };
};
```

Listing 4.3 – Modification des fichiers DTS

Le backend et frontend sont maintenant compilés et seront initialisés lors du démarrage de l'agence et de la ME. Le backend est toujours initialisé avant le frontend car il fait partie des drivers de l'agence. Ce n'est qu'une fois l'agence démarrée que la ME est à son tour lancée. Le backend et frontend vfb n'ont pas été écrits à partir de zéro mais sont basés sur ceux du vdummy, qui fournit une base fonctionnelle pour la création de drivers backend et frontend. Une fois le fichier `vdummy.c` copié, il suffit de remplacer toutes les occurrences de *vdummy* par *vfb*. Il ne faut pas oublier de faire de même pour les headers (`vdummy.h`).

4.3.2 Emplacement du framebuffer

Comme mentionné précédemment, il n'y a pas de notion de matériel dans une ME. C'est-à-dire que, bien que la plateforme ARM Versatile Express continue d'être utilisée, la ME n'a pas accès au contrôleur PL111, ni d'ailleurs à la VRAM mise à disposition par la plateforme.

Il est alors nécessaire de réfléchir à l'endroit en mémoire où une ME doit placer son framebuffer. Grâce à un mécanisme de SOO existant (les *grant tables* ou tables d'octrois), il serait possible de donner accès à la VRAM à une ME afin qu'elle y écrive l'interface graphique dessinée par l'application LittlevGL.

Cependant, cette solution pose deux problèmes. Premièrement, il faudrait mettre en place un système qui n'autoriserait que la ME active à aller écrire dans ce framebuffer. Sinon, chaque ME irait continuellement récrire ce que les autres ME ont écrit dans le framebuffer.

Deuxièmement, ce système empêcherait à l'agence d'accéder au framebuffer de chaque ME, indépendamment du domaine actif. En effet, parallèlement à ce travail de Bachelor est mené celui de Tommy Girardi [27], lequel nécessite un accès simultané à tous les framebuffers des ME virtualisées. Cela afin d'afficher sur un même écran divisé en plusieurs parties le contenu de chaque framebuffer. De ce fait, la solution proposée plus haut ne peut pas être considérée.

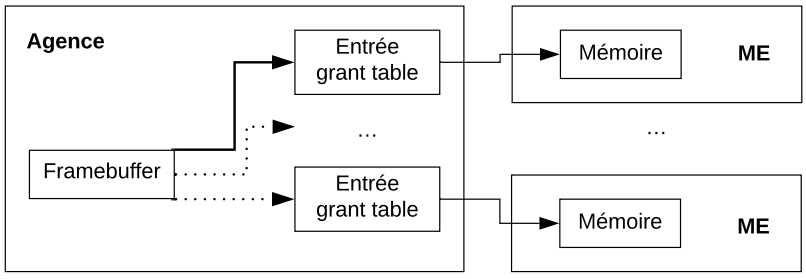


FIGURE 4.4 – Emplacement de la mémoire allouée pour le framebuffer : solution choisie

La solution choisie consiste à allouer un espace mémoire pour chaque framebuffer de ME. Cette allocation pourrait avoir lieu soit dans l'agence soit dans la ME. Même si, techniquement, les deux cas sont possibles, il a été décidé d'effectuer cette allocation dans la ME.

En effet, en allouant les espaces mémoire dans l'agence (c'est-à-dire par le noyau Linux), il devient plus difficile de protéger ces framebuffers, notamment d'écritures non désirées. Lorsque l'espace mémoire est alloué dans la ME, il est possible de définir l'accès donné à l'agence comme étant en lecture seule.

4.3.3 Taille du framebuffer

La taille du framebuffer dépend de la résolution de l'écran et du nombre de bits par pixel. Or, ces informations ne sont connues que par l'agence puisque c'est elle qui a accès au matériel. Il faut donc communiquer ces informations à la ME qui en a besoin afin d'allouer l'espace mémoire pour le framebuffer.

4.3.3.1 Obtention des propriétés du framebuffer

Le noyau Linux possède un grand nombre de drivers pour différents types de framebuffers (`drivers/video/fbdev`). Lorsque Linux détecte un contrôleur de framebuffer, il l'initialise et ensuite enregistre une référence du framebuffer dans le tableau `registered_fb` du fichier `fb.h`. Le tableau contient des pointeurs sur la structure `fb_info`. C'est une structure complexe qui est utilisée par Linux pour décrire et manipuler un framebuffer. Elle contient, entre autres, une structure de type `fb_var_screeninfo` qui contient notamment la résolution horizontale et verticale du framebuffer ainsi que le nombre de bits par pixel.

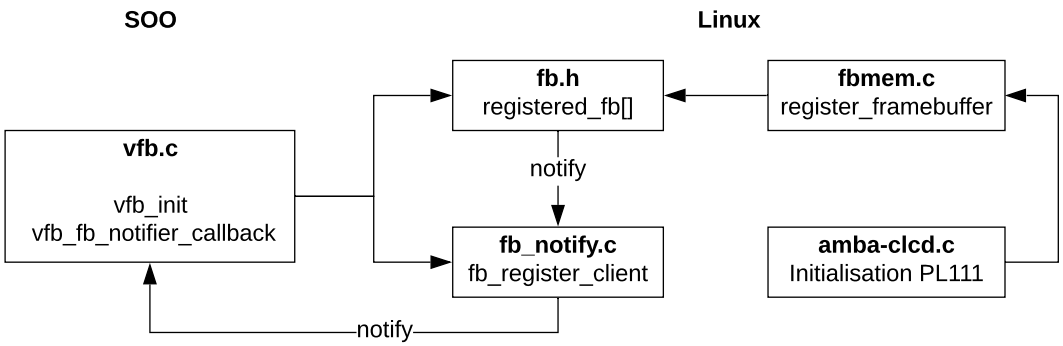


FIGURE 4.5 – Initialisation du backend vfb : recherche de la structure `fb_info`

Pour la plateforme Versatile Express, c'est le driver `amba-clcd.c` qui est utilisé par Linux pour initialiser le PL111. Cependant, le driver ne connaît pas les valeurs à utiliser pour initialiser le framebuffer. Ces valeurs proviennent de la DTS de la plateforme. L'extrait suivant est un nœud du fichier `vexpress-v2m-rs1.dtsi` décrivant le contrôleur PL111.

```
clcd@1f0000 {
    compatible = "arm,pl111", "arm,primecell";
    memory-region = <&vram>;

    panel {
        panel-timing {
            clock-frequency = <65000000>;
            hactive = <1024>;
            vactive = <768>;
            hsync-len = <136>;
            hfront-porch = <20>;
            hback-porch = <160>;
            vfront-porch = <3>;
            vback-porch = <29>;
            vsync-len = <6>;
        };
    };
};
```

Listing 4.4 – Description du PL111 dans la DTS de la plateforme Versatile Express

On peut y distinguer la résolution voulue, 1024×768, ainsi qu'une référence à la VRAM qui est décrite dans la DTS `vepxress-tz.dts`. Le nombre de bits par pixel est calculé par le driver à partir de la propriété `max-memory-bandwidth` qui n'est pas définie dans l'extrait précédent. Le driver choisi alors la valeur de 32 bits par pixel, ce qui nous convient.

Il s'agit maintenant de choisir le bon framebuffer du tableau `registered_fb`. Pour l'instant il n'est prévu d'avoir qu'un seul écran rattaché au système embarqué, choisir le premier framebuffer du tableau est une option suffisante. Cela est fait lors de l'initialisation du backend (`vfb_init`). La première structure `fb_info` non nulle de `registered_fb` est choisie pour décider du framebuffer qui sera utilisé. Elle est stockée dans une variable propre au backend nommée `vfb_info`. Une résolution minimale est cependant imposée afin d'exclure les framebuffers qui auraient une résolution trop petite. Celle-ci a été fixée arbitrairement à 640×480.

Le backend étant considéré comme un driver Linux au même titre que ceux des framebuffers, il n'est pas possible de garantir que le driver du framebuffer sera initialisé avant le backend, autrement dit que le tableau `registered_fb` ait été rempli avant sa lecture par le backend. Cependant, Linux met un place un système de notifications afin d'être notifier d'événements relatifs aux framebuffers. Il faut utiliser la fonction `fb_register_client` définie dans `fb_notify.c` et lui passer une structure de type `notifier_block`. Celle-ci contient une fonction callback qui sera appelée à chaque événement relatifs aux framebuffers.

Linux définit plusieurs événements de ce type mais ceux voulus ne sont disponibles que pour la plateforme AM200 EPD. Il faut donc modifier le fichier `fb.h` afin que les événements `FB_EVENT_FB_REGISTERED` et `FB_EVENT_FB_UNREGISTERED` soient disponibles pour toutes les plateformes. Les fonctions `do_register_framebuffer` et `do_unregister_framebuffer` de `fbmem.c` doivent aussi être modifiées pour que les événements soient bien envoyés. Les modifications consistent simplement à enlever les `#ifdef` utilisé par Linux pour restreindre le code à la plateforme AM200 EPD.

À noter qu'il n'est pas nécessaire de recevoir les événements relatif aux framebuffers si un framebuffer est détecté lors de l'initialisation du backend.

```
int vfb_init(void)
{
    /* We check if there's already a registered framebuffer device. */
    int i = 0;
    while (i < FB_MAX && vfb_set_agencyfb(registered_fb[i]))
        i++;

    /* Otherwise, we register a client that will be notified when a
     * framebuffer is registered. */
    if (i == FB_MAX)
        fb_register_client(&vfb_fb_notif);

    return 0;
}
```

Listing 4.5 – Fonction d'initialisation du backend vfb

4.3.3.2 Transmission des propriétés à la ME

Pour transmettre la résolution du framebuffer aux ME, il a été décidé d'utiliser VBStore et donc de créer un chemin dans lequel est créé une propriété pour la résolution horizontale et une autre pour la résolution verticale. VBStore se prête bien à cette utilisation car la résolution consiste en deux valeurs à transmettre uniquement à l'initialisation de la ME. Il n'y a pas besoin de mettre en place un anneau partagé, système plus complexe, juste pour transmettre les propriétés du framebuffer.

Chaque backend possède son propre espace VBStore et doit être initialisé dans la fonction `vbstorage_agency_init` du fichier `vbstorage.c`. L'extrait suivant montre comment il est créé.

```
np = of_find_compatible_node(NULL, NULL, "vfb,backend");
if (of_device_is_available(np))
    vbs_store_mkdir("/backend/vfb");
```

Listing 4.6 – Création du chemin `/backend/vfb` dans l'agence

Dans cet extrait, la présence du nœud contenant une propriété avec la valeur `vfb,backend` est cherché dans la DTS et s'il est trouvé, le chemin `/backend/vfb` est créé. Un chemin est créé de la même manière pour chaque ME et chaque frontend mais cela se passe dans le fichier `vbstore_me.c`. Une entrée doit être rajoutée pour le vfb comme le montre cet extrait de la fonction `vbstore_devices_populate`.

```
fdt_node = fdt_find_compatible_node("vfb,frontend");
if (fdt_device_is_available(fdt_node)) {
    DBG("%s: init vfb...\n", __func__);
    vbstore_dev_init(ME_domID(), "vfb", false, "vfb,frontend");
}
```

Listing 4.7 – Création du chemin `/devices/<domain-id>/vfb/0` dans le frontend

Cela crée un chemin ayant le format suivant `/devices/<domain-id>/vfb/0`. Puisque la résolution du framebuffer est commune à chaque ME, on pourrait l'écrire dans le VBStore du backend. Cependant, il a été choisi de l'écrire dans le VBStore de la ME car cela faciliterait un éventuel développement futur qui requerrait des résolutions différentes pour chaque ME.

Le chemin `devices/<domain-id>/vfb/0/resolution` est créé dans le backend avec la fonction `vbus_mkdir` définie dans `vbus_vbstore.c`. Puis, les deux propriétés `hor` et `ver` sont créées avec la fonction `vbus_printf`. Le code suivante montre comment cela a été fait.

```
struct vbus_transaction vbt;
char dir[40];

vbus_transaction_start(&vbt);

sprintf(dir, "device/%01d/vfb/0", vdev->otherend_id);
vbus_mkdir(vbt, dir, "resolution");

sprintf(dir, "device/%01d/vfb/0/resolution", vdev->otherend_id);
vbus_printf(vbt, dir, "hor", "%u", FB_HRES);
vbus_printf(vbt, dir, "ver", "%u", FB_VRES);

vbus_transaction_end(vbt);
```

Listing 4.8 – Création du chemin `resolution` et des propriétés `hor` et `ver`

Dans la fonction `vfb_probe` du frontend, la résolution est récupérée grâce à la fonction `vbus_scanf` définie dans `vbus_vbstore.c`.

```
struct vbus_transaction vbt;
uint32_t hres, vres;
char dir[40];

vbus_transaction_start(&vbt);
sprintf(dir, "device/%01d/vfb/0/resolution", ME_domID());
```

```
7 vbus_scanf(vbt, dir, "hor", "%u", &hres);
8 vbus_scanf(vbt, dir, "ver", "%u", &vres);
9 vbus_transaction_end(vbt);
```

Listing 4.9 – Frontend vfb : récupération de la résolution stockée dans le VBStore

4.3.4 Réservation de l'espace mémoire

Maintenant que la résolution est connue par le frontend, il est possible d'allouer l'espace mémoire pour le framebuffer puisque sa taille peut être calculée en multipliant la résolution horizontale, la résolution verticale et le nombre d'octet par pixel.

Il n'y a pas toujours une correspondance exacte entre le nombre de bits par pixel et le nombre d'octets par pixel. Par exemple, le PL111 définit le mode 24bpp mais un pixel sera quand même stocké sur 4 octets (un octet est ignoré). Dans Linux ainsi que LittlevGL, on parle plutôt de 32bpp (4 octets aussi). Le nombre de couleurs est le même (16 millions) mais au niveau logiciel le premier octet peut être utilisé pour la transparence.

Actuellement, le nombre d'octets par pixel a été fixé à quatre dans le backend. Si cela était amené à changer, il faudrait passer cette valeur au frontend via le VBStore comme pour la résolution.

Une fois la taille calculée, il est possible d'allouer l'espace mémoire pour le framebuffer. Cela est fait avec la fonction `get_contig_free_pages` qui réserve un certain nombre de pages physiques. Ces pages n'appartiennent à aucun processus et ne seront pas libérées, cela est très proche du fonctionnement de la VRAM.

```
1 uint32_t fb_base, hres, vres, page_count;
2 char dir[40];
3
4 page_count = hres * vres * 4 / PAGE_SIZE;
5 fb_base = get_contig_free_pages(page_count);
6 BUG_ON(!fb_base);
7 so3virt_fb_set_info(fb_base, hres, vres);
```

Listing 4.10 – Frontend vfb : réservation de l'espace mémoire pour le framebuffer

4.3.4.1 Création du driver virtuel

Le driver virtuel vient remplacer dans SOO.refso3 le driver du PL111 décrit dans le chapitre 3. Il a été écrit dans le fichier `so3virt_fb.c`. Comme pour les autres drivers, un nœud a du être rajouté dans la DTS `so3virt.dts`.

```
1 so3virt-fb {
```

```
    compatible = "fb,so3virt";
    status = "ok";
};
```

Listing 4.11 – Description du driver virtuel framebuffer dans la DTS de SOO.refso3

Il définit aussi les deux opérations sur fichier `mmap` et `ioctl` et dans sa fonction d'initialisation il enregistre une structure `classdev` afin que le fichier spécial `/dev/fb0` soit disponible dans l'espace utilisateur de la ME.

Les différences se trouvent dans la fonction d'initialisation, il n'y a pas de périphérique à initialiser et donc pas de registres à écrire. Dans la fonction `fb_mmap`, l'adresse physique de la VRAM y est remplacée par celle réservée par le frontend. Le frontend communique cette adresse grâce à la fonction `so3virt_fb_set_info` qui informe aussi le driver virtuel de la résolution de l'écran (nécessaire pour les opérations `ioctl`).

4.3.5 Mapping du framebuffer dans l'agence

Pour que l'espace mémoire réservé ci-dessus puisse être accédé par le backend, le frontend doit autoriser cet accès pour toutes les pages qui composent l'espace mémoire. Cela est fait juste après la réservation de l'espace mémoire par la fonction `gnttab_grant_foreign_access` qui prend comme paramètres l'identifiant du domaine de l'agence et la page à autoriser.

Cette fonction est définie dans `gnttab.c` (*grant table* ou table d'octrois) et fait partie d'un mécanise existant qui permet justement le partage de pages entre les différents domaines. La fonction retourne une référence vers l'entrée nouvellement créée dans la table d'octrois. C'est cette valeur qui est nécessaire au backend pour pouvoir accéder au framebuffer de la ME. La valeur étant un entier elle peut être écrite dans le chemin `device/%01d/vfb/0/domfb-ref` de VBStore.

```
for (i = 0; i < page_count; i++) {
    res = gnttab_grant_foreign_access(vdev->otherend_id,
                                      phys_to_pfn(fb_base + i * PAGE_SIZE),
                                      1);
    BUG_ON(res < 0);

    if (i == 0) /* The back-end only needs the first grantref. */
        fb_ref = res;
}

sprintf(dir, "device/%01d/vfb/0/domfb-ref", ME_domID());
vbus_printf(vbt, dir, "value", "%u", fb_ref);
```

Listing 4.12 – Frontend vfb : octroi de l'accès aux pages du framebuffer

Ce chemin a été préalablement créé par le backend dans la fonction `vbf_probe`. Le chemin

est aussi observé par le backend afin qu'il soit notifié de toute modification de propriétés. Lorsqu'il y a une modification, la fonction `callback_me_domfb` est appelée.

```
watches[vdev->otherend_id] = kzalloc(sizeof(struct vbus_watch), GFP_ATOMIC);
vbus_watch_pathfmt(vdev,
                   watches[vdev->otherend_id],
                   callback_me_domfb,
                   "device/%01d/vfb/0/domfb-ref/value",
                   vdev->otherend_id);
```

Listing 4.13 – Backend vfb : observation du chemin de la grantref

C'est cette fonction qui alloue un espace mémoire dans l'agence et le mappe vers l'espace réservé par le frontend (partagé via la table d'octrois). Le code source suivant illustre ce procédé.

```
grant_ref_t fb_ref;
struct gnttab_map_grant_ref op;
struct vm_struct *area;
domid_t domid; /* ME domain id */

area = alloc_vm_area(FB_SIZE, NULL);

gnttab_set_map_op(&op,
                  (phys_addr_t) area->addr,
                  GNTMAP_host_map | GNTMAP_readonly,
                  fb_ref, domid, 0, FB_SIZE);

gnttab_map(&op);
```

Listing 4.14 – Frontend vfb : allocation d'un espace mémoire pour le framebuffer

4.3.6 Affichage des différents framebuffers

Comme expliqué dans la section 4.2, il est possible de changer le domaine actif en tapant Ctrl + A deux fois. Avec les modifications apportées au fichier `console.c`, cette combinaison de touches appelle la fonction `vfb_set_active_domfb` du backend.

Le but de cette fonction est de modifier les registres du PL111, notamment le registre LCDUPBASE afin d'y mettre l'adresse physique du framebuffer à afficher. Il est possible de faire cela en modifiant certaines valeurs de la structure `fb_info` du framebuffer puis d'appeler l'opération `fb_set_par` qui va écrire les nouvelles valeurs de la structure dans les registres du contrôleur. L'appel de cette opération (définie dans la structure `fb_info`) exécutera la fonction `clcdfb_set_par` puis `clcdfb_set_start`, laquelle modifie les registres voulus.

La valeur de `fb_info` qui doit être modifiée est l'adresse physique de l'emplacement du framebuffer, soit `fb_info->fix.smem_start`. Par souci de cohérence, l'adresse virtuelle sera

aussi modifiée, elle se trouve dans `fb_info->screen_base`.

Afin d'enregistrer les adresses physiques et virtuelles des différents framebuffers, la structure `vfb_domfb` a été créée, elle représente un framebuffer de domaine. En effet, une telle structure ne sera pas seulement utilisée pour les ME mais aussi pour l'agence. Elle contient différentes valeurs :

- `domid_t id` : l'identifiant du domaine auquel appartient le framebuffer,
- `uint64_t paddr` : l'adresse physique communiquée par le frontend,
- `char *vaddr` : l'adresse virtuelle du framebuffer,
- `struct vm_struct *area` : un pointeur vers l'espace mémoire alloué par le backend,
- `grant_handle_t gnt_handle` : l'identifiant du mapping effectué par le backend.

Les deux dernières valeurs sont utiles lorsqu'une ME est terminée, cela permet de démapper l'espace alloué puis de le libérer. C'est la fonction `vfb_close` du backend qui s'occupe de libérer ces ressources.

Il y a donc une structure `vfb_domfb` pour chaque domaine devant afficher un framebuffer. Ces structures sont créées dans la fonction `callback_me_domfb` vue précédemment (à l'exception de la structure pour l'agence) et sont stockées dans le tableau `registered_domfb`. L'indice du tableau correspond à l'identifiant du domaine. La taille du tableau correspond donc au nombre maximal de domaines pouvant être virtualisés. Ce nombre est repris du fichier `console.c` et est fixé à 8 (à noter que l'hyperviseur est inclus dans ce nombre même si ce n'est pas un domaine et qu'il ne possède pas de framebuffer).

```
1  struct gnttab_map_grant_ref op;
2  struct vm_struct *area;
3  struct vfb_domfb *fb;
4  domid_t domid;
5
6  fb = kzalloc(sizeof(struct vfb_domfb), GFP_ATOMIC);
7  fb->id = domid;
8  fb->paddr = op.dev_bus_addr << 12;
9  fb->vaddr = area->addr;
10 fb->area = area;
11 fb->gnt_handle = op.handle;
12 vfb_set_domfb(fb); /* added to registered_domfb */
```

Listing 4.15 – Initialisation de la structure `vfb_domfb` d'une ME

L'agence possède aussi sa structure `vfb_domfb` car il est intéressant de pouvoir revenir sur le framebuffer de l'agence. Il est en effet possible que celui-ci ait été modifié par une autre application. La structure est créée dans la fonction `vfb_set_agencyfb` qui est appelée lorsque la structure `fb_info` a été détectée, c'est-à-dire lors l'initialisation du backend ou lorsqu'un framebuffer est enregistré par Linux et que le backend en est notifié.

```
static int vfb_set_agencyfb(struct fb_info *info)
{
    struct vfb_domfb *fb;

    if (vfb_info
        || !info
        || info->var.xres < MIN_FB_HRES
        || info->var.yres < MIN_FB_VRES) {
        return -1;
    }

    vfb_info = info;

    fb = kzalloc(sizeof(struct vfb_domfb), GFP_ATOMIC);
    fb->id = 0;
    fb->paddr = info->fix.smem_start;
    fb->vaddr = info->screen_base;

    vfb_set_domfb(fb);
    vfb_set_active_domfb(fb->id);

    return 0;
}
```

Listing 4.16 – Initialisation de la structure `vfb_domfb` pour l'agence

Il est nécessaire d'appeler `vfb_set_active_domfb` car bien que le PL111 soit détecté par Linux et que la structure `fb_info` soit initialisée, les registres du contrôleur n'ont pas été modifiés. L'appel est donc nécessaire pour que `clcdfb_set_par` soit appelé.

En activant le driver Linux *Framebuffer Console*, le contrôleur sera activé et l'appel à `vfb_set_active_domfb` ne sera pas nécessaire. Cependant, l'activation de ce driver pose problème car il crée un thread pour le clignotement du curseur de la console. Or, lorsque la structure `fb_info` est modifiée (lors d'un changement du domaine actif) le thread ne peut plus écrire dans l'espace mémoire et fait donc planter le noyau Linux. Ce driver n'étant pas requis, il a été décidé de le considéré comme toujours désactivé.

4.3.7 Notifications extérieures

Une partie du travail de Bachelor de Tommy Girardi requiert d'être informée lorsque le framebuffer d'une ME est créé puis détruit (suite à la terminaison de la ME). Puisque l'adresse virtuelle du framebuffer est nécessaire, il a été décidé d'implémenter un simple système de callback.

La fonction `vfb_set_callback_new_domfb` permet de définir la fonction à appeler lorsqu'une nouvelle entrée est rajoutée au tableau `registered_domfb`. La fonction doit avoir la signature suivante : `void (*cb)(struct vfb_domfb *, struct fb_info *)` et sera appelée

à la fin de la fonction `callback_me_domfb`.

La fonction `vfb_set_callback_rm_domfb` définit la fonction devant être appelée lorsqu'une ME est terminée, son prototype est `void (*cb)(domid_t id)` et elle sera appelée dans la fonction `vfb_close` du backend.

4.3.8 Terminaison d'une ME

La terminaison d'une ME implique que plusieurs ressources doivent être libérées.

- La structure `vbus_watch` utilisée pour être notifié des modifications du chemin contenant la `grant_ref` doit être libérée après avoir été desenregistrée.
- L'espace mémoire alloué par le backend doit être dé-mappé puis libéré (structure `vm_struct`).
- L'entrée `vfb_domfb` doit aussi être supprimée du tableau et libérée.

```
1  void vfb_close(struct vbus_device *vdev)
2  {
3      struct gnttab_unmap_grant_ref op;
4
5      /* Unregister and free the watch. */
6      unregister_vbus_watch(watches[vdev->otherend_id]);
7      kfree(watches[vdev->otherend_id]);
8      watches[vdev->otherend_id] = NULL;
9
10     /* Unmap the grantref. */
11     gnttab_set_unmap_op(
12         &op,
13         (phys_addr_t) registered_domfb[vdev->otherend_id]->vaddr,
14         GNTMAP_host_map | GNTMAP_readonly,
15         registered_domfb[vdev->otherend_id]->gnt_handle);
16     gnttab_unmap(&op);
17
18     /* Free framebuffer area. */
19     free_vm_area(registered_domfb[vdev->otherend_id]->area);
20
21     /* Free vfb_domfb. */
22     kfree(registered_domfb[vdev->otherend_id]);
23     registered_domfb[vdev->otherend_id] = NULL;
24
25     if (callback_rm_domfb) {
26         callback_rm_domfb(vdev->otherend_id);
27     }
28 }
```

Listing 4.17 – Backend vfb : libération des ressources lorsqu'une ME est terminée

4.4 Gestion des périphériques d'entrée

4.4.1 Événements d'entrée

Le module *input* est un module du noyau Linux par lequel passe tous les événements d'entrée, qu'ils proviennent d'une souris, d'un clavier, d'un écran tactile ou encore d'autres périphériques d'entrée. L'existence de ce module a grandement simplifié l'écriture du backend et frontend des périphériques d'entrée, nommé vinput.

En effet, Linux possédant déjà de multiples drivers pour les périphériques d'entrée, il n'est pas nécessaire de se préoccuper des particularités de chacun, comme cela a été fait avec le PL050. De plus, pour chaque événement reçu, le driver du périphérique appelle la fonction `input_event` définie dans le module input de Linux (`input.c`). Un tel événement [6] est composé de trois valeurs (déclarées dans `include/uapi/linux/input-event-codes.h`) :

- `unsigned int type` : désigne le type d'événement, il en existe une douzaine mais ceux qui sont intéressants pour ce travail sont `EV_KEY` (e.g. touches de clavier, boutons de souris), `EV_REL` (déplacement de la souris) et `EV_ABS` (déplacement de l'écran tactile).
- `unsigned int code` : un code identifiant plus précisément l'événement, par exemple le code 42 pour le type `EV_KEY` désigne la touche du clavier *Left Shift*, le code 0 pour le type `EV_REL` désigne le déplacement de la souris selon l'axe des abscisses.
- `int value` : la valeur de l'événement, pour une touche clavier, indique si elle a été appuyée ou relâchée, pour un déplacement de la souris c'est la valeur de ce déplacement qui est indiqué.

Afin de récupérer ces événements dans le backend vinput, il suffit de modifier la fonction `input_event` afin d'appeler la fonction `vinput_pass_event` définie dans le backend qui s'occupera de transmettre l'événement au frontend.

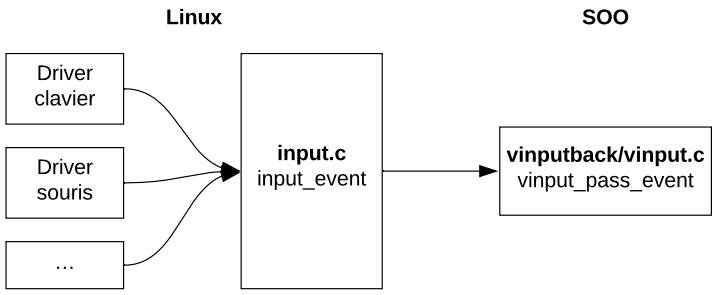


FIGURE 4.6 – Transmission des événements d'entrée des drivers Linux vers le backend vinput

4.4.2 Création du backend et frontend

Le processus de création du backend et du frontend est très similaire à celui du framebuffer. Il est basé sur vdummy et a été placé dans le répertoire `vinput`. Il faut créer un nouveau nœud dans la DTS Linux et celle de la ME, ainsi que modifier les fichiers Kconfig correspondants afin que les drivers puissent être activés ou désactivés dans la configuration du noyau.

La différence principale entre le vfb et vinput se trouve dans la communication entre le backend et le frontend. Dans vfb, trois propriétés dans VBStore ont suffit. Par contre, utiliser VBStore pour l'envoi d'événements n'est pas adapté car il est possible qu'ils soient produits (par l'agence) plus vite qu'ils ne soient consommés (par la ME).

Il est plus adapté d'utiliser un anneau partagé entre le backend et le frontend. Cet anneau permet au backend de déposer des réponses sur l'anneau, le frontend peut ensuite lire les réponses à sa guise. Le backend notifie le frontend via une interruption, ce qui est très similaire au fonctionnement du PL050 et des périphériques d'entrée en général. De la même manière, le frontend peut déposer des requêtes sur l'anneau et notifier le backend. À noter qu'une réponse ne présuppose pas une requête, les deux notions sont indépendantes.

Dans le cas du vinput, seules les réponses sont nécessaires car les événements d'entrées ne sont transmis que du backend vers le frontend. Une réponse correspond à une structure. Afin de transmettre les événements d'entrée, la structure `vinput_response_t` a été déclarée telle que :

```
1 typedef struct  {
2     unsigned int type;
3     unsigned int code;
4     int value;
5 } vinput_response_t;
```

Listing 4.18 – Backend vinput : structure de la réponse

4.4.3 Gestion des ME

Lorsqu'un frontend est connecté, la fonction `vinput_connected` du backend est exécutée. Cette fonction a été modifiée pour garder une référence des frontend connectés. Un tableau a été créé à cet effet et contient un pointeur sur une structure `vinput_t`. L'indice du tableau correspond à l'identifiant du domaine.

Lorsque le domaine actif change suite à un double Ctrl + A, `vinput_set_current` est appelée. Cette fonction enregistre simplement l'identifiant du domaine actif passé en paramètre dans la variable `current_vinput`.

Ainsi, afin de savoir à quel frontend transmettre les événements d'entrée, il suffit d'ac-

céder à l'élément `current_vinput` du tableau `vinputs`. Il est important de restreindre les événements d'entrée à la seule ME active, car ces événements proviennent de l'utilisateur qui voit à l'écran l'interface graphique de la ME active.

4.4.4 Transmission des événements

La figure 4.8 montre l'utilisation de l'anneau partagé afin de transmettre les événements d'entrée du backend au frontend et ensuite vers les drivers virtuels.

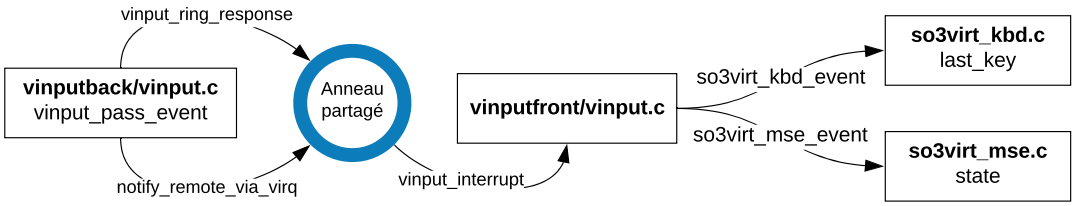


FIGURE 4.7 – Transmission des événements d'entrée du backend au frontend via l'anneau partagé

4.4.4.1 Du backend au frontend

La fonction `vinput_pass_event`, appelée par le module input de Linux, fait usage de l'anneau partagé afin de transmettre les événements au frontend.

```
1  void vinput_pass_event(unsigned int type, unsigned int code, int value)
2  {
3      vinput_response_t *ring_rsp;
4      vinput_t *vinput = vinputs[current_vinput];
5
6      /* If the frontend is not connected, skip. */
7      if (!vinput)
8          return;
9
10     /* Send event to front-end. */
11     ring_rsp = vinput_ring_response(&vinput->ring);
12     ring_rsp->type = type;
13     ring_rsp->code = code;
14     ring_rsp->value = value;
15
16     vinput_ring_response_ready(&vinput->ring);
17     notify_remote_via_virq(vinput->irq);
```

```
}
```

Listing 4.19 – Backend vinput : envoi d'une réponse au frontend

La fonction `notify_remote_via_irq` permet, comme son nom l'indique, de notifier le frontend de l'existence d'une nouvelle réponse via une interruption. Cela aura pour effet d'exécuter la fonction `vinput_interrupt` du frontend, puisqu'elle a été définie comme étant le gestionnaire d'interruptions. L'extrait de code suivant montre comment récupérer la réponse du backend.

```
irq_return_t vinput_interrupt(int irq, void *dev_id)
{
    struct vbus_device *vdev = (struct vbus_device *) dev_id;
    vinput_t *vinput = to_vinput(vdev);
    vinput_response_t *ring_rsp;
    unsigned int type, code;
    int value;

    while ((ring_rsp = vinput_ring_response(&vinput->ring)) != NULL) {

        type = ring_rsp->type;
        code = ring_rsp->code;
        value = ring_rsp->value;

        /* Handle input event. */
    }

    return IRQ_COMPLETED;
}
```

Listing 4.20 – Backend vinput : envoi d'une réponse au frontend

4.4.4.2 Du frontend aux drivers virtuels

Deux drivers virtuels ont été créés dans la ME : `so3virt_kbd.c` pour la clavier et `so3virt_mse.c` pour la souris. Leur but est la création d'un fichier `/dev/keyboard0` et `/dev/mouse0` depuis l'espace utilisateur mais aussi de recevoir et traiter les événements reçus du frontend. Puisque ces drivers n'interagissent pas avec un contrôleur de périphérique comme le PL050, le protocole PS2 n'est plus utilisé, simplifiant grandement ces drivers.

Afin qu'aucune modification ne soit nécessaire dans l'espace utilisateur, les drivers doivent réagir aux mêmes opérations `ioctl` définies dans le driver du PL050. C'est-à-dire retourner la dernière touche appuyée pour le driver du clavier et retourner les coordonnées de la souris pour le driver de la souris.

Driver clavier Dans le driver du clavier, la fonction `so3virt_kbd_event` a été créée et permet au frontend de passer les événements clavier au driver virtuel. Dans cette fonction, le driver doit transformer les événements d'entrée en un caractère UTF-8. Il faut détecter si la touche Shift est appuyé lorsqu'un événement correspondant à une lettre est reçu. Si c'est le cas alors il faut retourner la lettre majuscule correspondante.

À noter que si une touche est relâchée, sa valeur n'est pas mise à zéro dans la fonction `so3virt_kbd_event`, mais seulement une fois qu'elle a été lue par un appel `ioctl`. Cela afin d'être sûr que le client ait le temps de lire la touche. Un système plus robuste aurait pu être implémenté en utilisant une file où les événements seraient lus par le driver virtuel lors d'un appel `ioctl`.

S'il avait été nécessaire de gérer des touches (ou combinaison de touches) qui ne retournent pas un caractère mais qui exécutent une action (e.g. raccourci clavier), il aurait été plus judicieux de gérer les événements dans l'espace utilisateur, ces actions étant propres à chaque application.

Driver souris Dans le driver de la souris, la fonction `so3virt_mse_event` a été créée et permet au frontend de passer les événements relatif à la souris au driver virtuel. Dans cette fonction, les coordonnées absolues de la souris doivent être calculées et les boutons de la souris doivent être enregistrés.

Les événements d'entrée de type `EV_REL` n'envoie pas les coordonnées absolues de la souris mais le déplacement de celle-ci par rapport à l'événement précédent et selon un axe (abscisse ou ordonnée). Il faut alors calculer les coordonnées absolues de la souris tout comme cela est fait pour le driver PL050.

Séparation des événements clavier–souris Dans la fonction `vinput_interrupt` le frontend doit déterminer quels événements sont envoyés au clavier et lesquels à la souris. Cela a été fait en regardant le type de l'événement mais aussi le code.

```
1  while ((ring_rsp = vinput_ring_response(&vinput->ring)) != NULL) {
2
3      type = ring_rsp->type;
4      code = ring_rsp->code;
5      value = ring_rsp->value;
6
7      if (type == EV_REL || /* is mouse movement */
8         (type == EV_KEY && /* is mouse button click */
9           (code == BTN_LEFT || code == BTN_MIDDLE || code == BTN_RIGHT)) {
10          so3virt_mse_event(type, code, value);
11     }
12     else if (type == EV_KEY) {
13         so3virt_kbd_event(type, code, value);
14     }
```

```
15
16      /* Skip unhandled events. */
17  }
```

Listing 4.21 – Frontend vinput : tri des événements clavier–souris

4.5 Portage sur Raspberry Pi 4

4.5.1 Introduction

Jusqu'à présent, autant SO3 que SOO ont été testés sur la plateforme ARM Versatile Express émulée par QEMU. Cela a été fait pour faciliter et accélérer le développement mais cette situation ne correspond pas à la réalité d'une mise en production de SOO.

Le déploiement de SOO sur la Raspberry Pi 4 [19] est un objectif de ce travail de Bachelor. Ce déploiement permet de tester le travail réalisé dans de vraies conditions et en utilisant un périphérique d'entrée comme un écran tactile sur lequel est aussi affiché l'interface graphique.

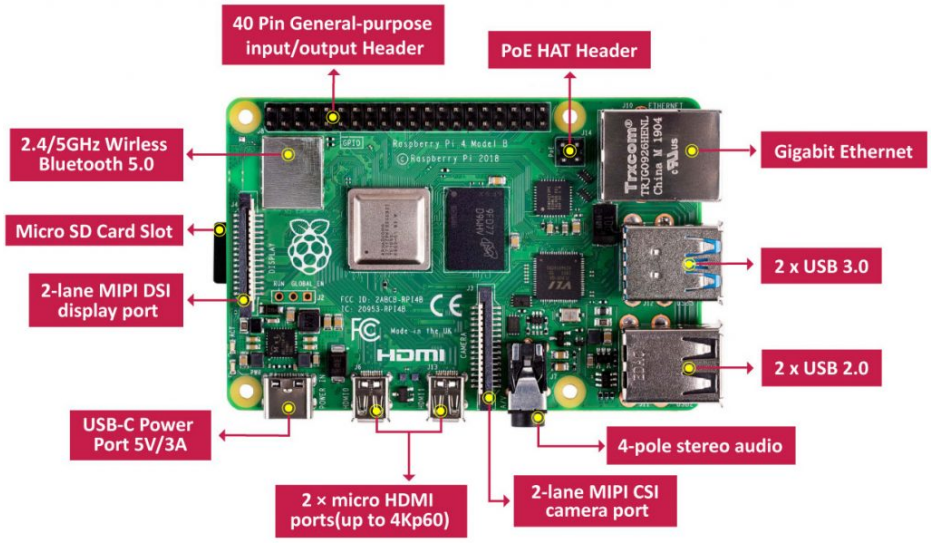


FIGURE 4.8 – Composants de la Raspberry Pi 4 [18]

4.5.2 Écrans tactiles

Trois écrans tactiles ont été achetés par le REDS afin de pouvoir les tester sur la Pi 4 et aussi de comparer les différences entre ces écrans, notamment la taille et résolution mais aussi

la manière dont ils se connectent à la Pi 4 (e.g. HDMI, DSI). Après recherche et comparaison de multiples écrans, ceux choisis sont :

1. l'écran officiel de la fondation Raspberry Pi d'une taille de 7" et d'une résolution de 800×480,
2. un écran de la compagnie 52Pi d'une taille de 7" aussi mais d'une résolution plus élevée de 1024×600,
3. un écran de la compagnie Waveshare d'une taille de 10.1" et d'une résolution de 1280×800.

Le premier écran se connecte à la Pi 4 sur son port DSI. L'alimentation de l'écran peut être faite grâce au port micro-USB ou via les ports GPIO. Les événements d'entrée sont transférés via le port DSI.

Le deux derniers écrans se connectent sur le port micro-HDMI de la Pi 4. Leur alimentation ne peut être faite que par un port micro-USB présent sur chacun des écrans.

4.5.3 Gestion des événements tactiles

4.5.3.1 Fonctionnement avec Raspbian

Avec l'OS Raspbian — distribution officiel de la fondation Raspberry Pi basée sur Debian — les trois écrans sont reconnus automatiquement. Aucune configuration de l'OS n'est nécessaire.

4.5.3.2 Fonctionnement avec SOO

Les écrans n'ont pas fonctionné du premier coup lorsque la Pi 4 démarrait avec SOO. Le premier problème est qu'ils ne s'allumaient pas, bien que le branchement fut le même. La raison était que les ports USB n'étaient pas détectés par SOO et que, par conséquent, les écrans n'étaient pas alimentés.

Plusieurs solutions ont été testées mais sans succès. D'abord, différents drivers ont été activés dans le noyau Linux, notamment ceux relatifs aux contrôleurs USB, xHCI et PCIe. Puis, les fichiers DTS de SOO pour la Pi 4 ont été comparés avec ceux de la dernière version du noyau Linux, lequel offre un meilleur support de la Pi 4. Les fichiers DTS les plus récents n'ont pas pu être importés tels quels car ceux de SOO comportent plusieurs modifications relatives à SOO. Les fichiers ont alors été comparés un à un et les modifications qui auraient pu avoir un lien avec les ports USB ont été intégrée à SOO, notamment un bloc relatif au bus PCIe, ce dernier étant utilisé par les ports USB.

Après une recherche plus approfondie du Prof. Daniel Rossier, il semblerait que les adresses utilisées par le bus PCIe soient sur 64 bits alors que l'OS est compilé en 32 bits. Il

faudrait donc que SOO soit compilé avec l'option LPAE (*Large Physical Address Extensions*) pour que les ports USB fonctionnent.

Le non support des ports USB par SOO a amené un deuxième problème concernant les événements d'entrée des écrans tactile. En effet, alors que les câbles USB alimentant les écrans peuvent être branchés — à des fins de test — sur un ordinateur à proximité, les événements devant être transmis via USB ne peuvent alors pas être reçu par SOO. Une souris ou un clavier ne peuvent pas non plus être branchés.

Par conséquent, sur la Pi 4, les différents périphériques d'entrée n'ont pas été testés lors de ce travail de Bachelor.

4.5.3.3 Implémentation du support des événements tactiles

Malgré l'impossibilité de tester les événements tactiles, il a quand été décider d'ajouter un support basique de ceux-ci. Ces événements sont aussi transmis par les différents drivers à la fonction `input_event` mentionnée précédemment et se composent donc aussi des trois valeurs `type`, `code` et `value`.

Événements absolus Le support de deux types d'événements envoyés par l'écran a été décidé. Le premier concerne les événements de type `EV_ABS` permettant de déplacer le curseur à l'écran. Contrairement à la souris, les événements possèdent une abscisse et une ordonnées. Par contre, ces coordonnées ne sont pas relatives à la résolution de l'écran. Lorsque l'écran tactile et connecté à la Pi 4, il communique au driver une valeur minimale et maximale pour chacun des deux axes et les coordonnées y sont relatives.

Cette valeur peut être récupérée dans la fonction `input_register_device` du fichier `input.c`. Depuis cette fonction est appelée une nouvelle fonction `vinput_set_touch_bounds` créée dans le backend. Les valeurs minimales et maximales sont récupérées et enregistrées dans deux tableaux du frontend comme illustré dans les extraits de codes suivants.

```
1  int input_register_device(struct input_dev *dev)
2  {
3      /* ... */
4
5      if (test_bit(EV_ABS, dev->evbit) && dev->absinfo) {
6          vinput_set_touch_bounds(dev->absinfo);
7      }
8
9      /* ... */
10 }
```

Listing 4.22 – `input.c` : envoi des valeurs minimales et maximales des écrans tactiles vers le backend

```
static int32_t min[] = {0, 0};
static int32_t max[] = {1000, 1000}; /* Test values. */

void vinput_set_touch_bounds(struct input_absinfo *info)
{
    min[0] = info[0].minimum; /* x */
    max[0] = info[0].maximum;
    min[1] = info[1].minimum; /* y */
    max[1] = info[1].maximum;
}
```

Listing 4.23 – Backend vinput : enregistrement des valeurs minimales et maximales des écrans tactiles

Ces valeurs ne sont pas transmises au frontend mais sont utilisées directement par le backend pour modifier les événements `EV_ABS` afin de ramener les coordonnées à la résolution de l'écran. Cela est fait dans la fonction `vinput_pass_event` du backend. Ainsi, le frontend reçoit directement les coordonnées de l'événement et peut les transmettre telles quelles au driver virtuel qui modifiera ensuite la position du curseur.

Événements « touch » L'écran émet aussi un événement lorsque l'utilisateur tape du doigt sur l'écran. Dans ce cas c'est un événement de type `EV_KEY` avec le code `BTN_TOUCH` qui est envoyé. Actuellement, un tel événement sera envoyé au driver du clavier par le frontend. Il faut cependant qu'il soit envoyé à celui de la souris afin de simuler un clique avec le bouton gauche de la souris. La condition envoyant les événements au driver de la souris et modifiée comme suit :

```
if (type == EV_REL || /* is mouse movement */
   (type == EV_KEY && /* is mouse button click */
    (code == BTN_LEFT || code == BTN_MIDDLE || code == BTN_RIGHT || code ==
    BTN_TOUCH)) ||
   (type == EV_ABS && /* is touchscreen event */
    (code == ABS_X || code == ABS_Y))) {

    so3virt_mse_event(type, code, value);
}
```

Listing 4.24 – Envoi des événements « touch » au driver de la souris

4.5.4 Gestion des framebuffers

4.5.4.1 Détection de la résolution

Avec Raspbian, la résolution des écrans est détectée automatiquement. Avec SOO, il est nécessaire de communiquer cette résolution dans le fichier `config.txt`. Ce fichier agit comme

le BIOS sur un ordinateur du bureau [17]. Communiquer la résolution implique qu'il faille modifier le fichier lorsque l'on change d'écran.

L'extrait suivant montre la configuration nécessaire pour les deux derniers écrans. Pour l'écran officiel de Raspberry, aucune modification n'est nécessaire. En effet, les modifications suivantes ne concernent que les écrans HDMI.

```
# 52Pi - 7"
hdmi_group=2
hdmi_mode=87
hdmi_cvt=1024 600 60

# Waveshare - 10.1"
hdmi_group=2
hdmi_mode=27
```

Listing 4.25 – Configuration du `config.txt` pour les écrans tactiles connectés via HDMI

Un mode correspond à une résolution et une fréquence d'affichage. Si la résolution voulue n'existe pas, il faut créer un nouveau mode et indiquer la résolution ainsi que la fréquence manuellement avec la propriété `hdmi_cvt`. C'est qui est fait pour l'écran 52Pi.

4.5.4.2 Création du framebuffer

Le framebuffer de la Pi 4 est géré par un processeur multimédia nommé *VideoCore* et développé par Broadcom. La documentation sur ce processeur est extrêmement limitée, surtout pour la version 6 qui est très différente de la version 4 embarquée sur la Raspberry Pi 3 [25].

Lorsque la résolution est correctement mise dans le `config.txt`, le framebuffer est correctement créé par Linux et est présent dans le tableau `registered_fb`. Cependant, il a été remarqué que la fonction `fb_set_par` de la structure `fb_info` n'était pas définie et donc qu'il était impossible de modifier l'adresse du framebuffer devant être affiché à l'écran.

Afin d'avoir plus d'information sur le driver utilisé pour le framebuffer du VideoCore, la fonction `dump_stack` a été utilisée dans la fonction `register_framebuffer` de `fbmem.c`. Elle permet d'afficher la trace d'appels ayant mené à cette fonction. Il a été découvert que le driver utilisé est un driver générique nommé *Simple Framebuffer*. Il semblerait alors que l'adresse mémoire physique du framebuffer est déterminée par le VideoCore et ne puisse pas être changée.

Cela pose un problème car le système implémenté jusqu'à présent repose sur la précondition que l'adresse du framebuffer puisse être changée.

4.5.4.3 Affichage des différents framebuffers – Adaptation

La solution implémentée afin de palier cette différence fut d'implémenter un thread copiant périodiquement le framebuffer du domaine actif dans le framebuffer du VideoCore, mais seulement quand le domaine actif est une ME.

Par contre, cela aura pour effet d'effacer le framebuffer de l'agence. Il faut donc allouer un espace mémoire dédié au framebuffer de l'agence. Cet espace mémoire ne sera utilisé que pour sauvegarder et restaurer l'état du VideoCore lorsque l'agence est le domaine actif. Cette allocation est faite dans la fonction `vfb_set_agencyfb`. L'extrait suivant montre la création de la structure `vfb_domfb` pour l'agence ainsi que l'allocation mémoire.

```
static int vfb_set_agencyfb(struct fb_info *info)
{
    struct vfb_domfb *fb;
    fb = kzalloc(sizeof(struct vfb_domfb), GFP_ATOMIC);
    fb->id = 0;

#ifdef CONFIG_ARCH_VEXPRESS
    /* ... */
#endif

#ifdef CONFIG_ARCH_BCM2835 /* Pi 4 */
    fb->vaddr = vmalloc(FB_SIZE);

    vfb_set_domfb(fb);
    vfb_set_active_domfb(fb->id);

    /* Start the framebuffer copy thread. */
    BUG_ON(!kthread_run(kthread_domfb_cpy, NULL, "fb-copy-thread"));
#endif

    return 0;
}
```

Listing 4.26 – Allocation de l'espace mémoire pour le framebuffer de l'agence

La sauvegarde de l'état du VideoCore dans le framebuffer de l'agence ne doit être fait seulement lorsque l'on passe de l'agence à une ME (via le double Ctrl + A). Cette sauvegarde est donc faite dans la fonction `vfb_set_active_domfb` grâce au code suivant :

```
void vfb_set_active_domfb(domid_t domid)
{
    /* ... */

    if (active_domfb == 0 && domid != 0) {
        memcpy(registered_domfb[0]->vaddr, vfb_info->screen_base, FB_SIZE);
    }
```

```
}
```

Listing 4.27 – Sauvegarde du framebuffer du VideoCore dans celui de l'agence

Puis, la restauration du framebuffer de l'agence dans celui du VideoCore ce fait dans le thread de copie, mais seulement lorsque l'on passe d'une ME à l'agence.

```
static int kthread_domfb_cpy(void *arg)
{
    void *addr_domfb, *addr_vc = vfb_info->screen_base;
    uint32_t previous_domfb = 0;

    while (true) {
        if (active_domfb != 0 || previous_domfb != 0) {
            previous_domfb = active_domfb;
            addr_domfb = registered_domfb[previous_domfb]->vaddr;
            memcpy(addr_vc, addr_domfb, FB_SIZE);
        }

        msleep(50);
    }

    return 0;
}
```

Listing 4.28 – Thread de copie périodique des framebuffers

Même si cette solution n'est utilisée pour le moment que sur la Pi 4, son avantage est qu'elle pourrait aussi l'être pour un autre contrôleur qui ne permet pas le changement de l'adresse du framebuffer. Il est aussi tout à fait possible d'utiliser cette solution pour tous les contrôleurs de framebuffer même si dans certains cas les performances seront dégradées à cause du thread de copie. Cependant, cela permettrait de ne pas avoir à gérer plusieurs architecture comme c'est maintenant le cas.

Chapitre 5

Conclusion

Ce travail de Bachelor, en ajoutant le support graphique dans SO3, a demandé des recherches à plusieurs niveaux du développement de systèmes d'exploitation. D'abord, il a fallu comprendre le fonctionnement et le but d'un contrôleur. Cela a permis l'implémentation de drivers assurant la communication entre les contrôleurs et le système d'exploitation. Le développement de driver a demandé des compétences diverses comme le fonctionnement d'un framebuffer ou la compréhension du protocole PS/2 et son utilisation pour communiquer avec la souris et le clavier. Il a ensuite été nécessaire de passer du noyau à l'espace utilisateur avec la modification d'appels système mais aussi avec l'intégration d'une librairie et l'écriture d'une application l'utilisant.

De nombreuses connaissances ont aussi été acquises autour des différents composants de la technologie SOO comme l'hyperviseur AVZ, les anneaux partagés, le VBStore et les drivers backend et frontend. De plus, une compréhension basique du fonctionnement des drivers Linux a pu être assimilée. Ce travail de Bachelor a aussi été une introduction aux systèmes embarqués et aux multiples possibilités qu'ils offrent.

Cependant, différents développements devront encore être entrepris afin qu'un utilisateur soit capable d'interagir avec une Raspberry Pi 4 au seul moyen d'un écran tactile. Le support des ports USB doit être implémenté et le support des événements d'entrée de l'écran tactile doit être testé en condition réelle. Diverses améliorations peuvent aussi être apportées à la ME SOO.refso3, comme un driver clavier plus robuste permettant de supporter autant des caractères UTF-8 que des raccourcis clavier définis par l'application.

Finalement, l'écriture d'applications concrètes devra être menée dans différents domaines tels que la domotique, l'affichage collaboratif ou encore le pilotage de drones. Pour cela, la migration de la ME développée jusqu'ici et son interaction avec d'autres devra être concrétisé.

Bibliographie

[1] ARM Versatile Express – PL050. https://developer.arm.com/documentation/dui0447/j/programmers-model/io-peripherals-and-interfaces/keyboard-and-mouse-interface--kmi?lang=en.

[2] ARM Versatile Express – PL111. https://developer.arm.com/documentation/dui0447/j/programmers-model/io-peripherals-and-interfaces/color-lcd-controller?lang=en.

[3] ARM – PL050 Technical Reference Manual. https://developer.arm.com/documentation/ddi0143/c/.

[4] ARM – PL111 Technical Reference Manual. https://developer.arm.com/documentation/ddi0293/c/.

[5] Light and Versatile Embedded Graphics Library. https://lvgl.io.

[6] Linux Documentation – Input event codes. https://www.kernel.org/doc/html/latest/input/event-codes.html.

[7] LittlevGL Demo App Example. https://github.com/littlevgl/lv_examples/blob/master/lv_tests/lv_test_theme/lv_test_theme_1.c.

[8] LVGL – Display interface. https://docs.lvgl.io/v7/en/html/porting/display.html.

[9] LVGL – File system. https://docs.lvgl.io/v7/en/html/overview/file-system.html.

[10] LVGL – Initialisation. https://docs.lvgl.io/latest/en/html/get-started/quick-overview.html#add-lvgl-into-your-project.

[11] LVGL – Input device interface. https://docs.lvgl.io/v7/en/html/porting/indev.html.

[12] LVGL – Task handler. https://docs.lvgl.io/latest/en/html/porting/task-handler.html.

[13] LVGL – Tick interface. https://docs.lvgl.io/latest/en/html/porting/tick.html.

[14] musl libc. https://musl.libc.org.

[15] OSDev Wiki – PS/2 Keyboard. https://wiki.osdev.org/PS/2_Keyboard.

[16] QEMU User Documentation – 3.5 ARM System Emulator. https://www.qemu.org/docs/master/qemu-doc.html#ARM-System-emulator.

[17] Raspberry Pi 4 – config.txt. https://www.raspberrypi.org/documentation/configuration/config-txt/README.md.

[18] Raspberry Pi 4 – Schematics. https://www.waveshare.com/img/devkit/Raspberry-Pi-4-Model-B/Raspberry-Pi-4-Model-B-intro.jpg.

[19] Raspberry Pi Foundation. https://www.raspberrypi.org.

[20] SO3 – Smart Object Oriented Operating system. https://gitlab.com/smartobject/so3.

[21] SOO – Smart Object Oriented. https://gitlab.com/smartobject/soo.

[22] Wikipedia – Device file. https://en.wikipedia.org/wiki/Device_file.

[23] Wikipedia – Framebuffer. https://en.wikipedia.org/wiki/Framebuffer.

[24] Wikipedia – ioctl. https://en.wikipedia.org/wiki/Ioctl.

[25] Wikipedia – VideoCore datasheets available only under NDA. https://en.wikipedia.org/wiki/VideoCore#Data_sources.

[26] Daniel Rossier. Smart Object Oriented (SOO) Technology. http://blog.reds.ch/?p=1020.

[27] Tommy Girardi. Smart Object Oriented – SOO.display Système d'affichage collaboratif. https://wiki.osdev.org/PS/2_Keyboard.

Table des figures

2.1 Exemple d'interface graphique tel que présenté sur le site web de LittlevGL . 3
2.2 Arborescence de `so3/usr/` 4
2.3 Vue générale de la compilation d'une application SO3 5
2.4 Vue générale d'une application LittlevGL et de son interaction avec SO3 . . . 7
2.5 Vue générale des drivers LittlevGL et de leurs interactions avec SO3 12
2.6 Premier onglet de l'application de démonstration avec affichage d'une image . 13
2.7 Deuxième onglet de l'application de démonstration avec un champ texte . . . 14

3.1 Fenêtre QEMU pour l'affiche de l'interface graphique 16
3.2 Interactions avec le VFS lors de l'ouverture d'un fichier 18
3.3 Vue général de l'implémentation des fichiers spéciaux dans SO3 20
3.4 Registres du PL111 tels que décrits dans la documentation ARM 21
3.5 L'adresse physique de la VRAM est `0x18000000` 22
3.6 Détail des bits du registre LCDControl [4] 23
3.7 Vue générale du mapping de la mémoire du framebuffer 25
3.8 Registres du PL050 tels que décrits par la documentation ARM [3] 28

4.1 Interactions entre les drivers Linux, ceux du backend et frontend, et les drivers virtuels 34
4.2 Exemple d'utilisation du double Ctrl + A 35
4.3 Activation du vfb dans le menu Kconfig de l'agence 37
4.4 Emplacement de la mémoire allouée pour le framebuffer : solution choisie . . . 38
4.5 Initialisation du backend vfb : recherche de la structure `fb_info` 39

4.6 Transmission des événements d'entrée des drivers Linux vers le backend vinput 49
4.7 Transmission des événements d'entrée du backend au frontend via l'anneau partagé 51
4.8 Composants de la Raspberry Pi 4 [18] 54

Listings

2.1 `usr/Makefile` : ajout de la cible compilant la librairie . . . 5
2.2 Exemple d'application LittlevGL . . . 6
2.3 Exemple d'écriture dans un framebuffer . . . 8
2.4 LittlevGL : initialisation du driver d'affichage . . . 9
2.5 Fonction callback de copie du buffer LittlevGL dans le framebuffer . . . 9
2.6 LittlevGL : initialisation du driver pour la souris . . . 10
2.7 Fonction callback pour récupérer les coordonnées de la souris . . . 10
2.8 Fonction callback pour récupérer la dernière touche appuyée du clavier . . . 11
2.9 LittlevGL : initialisation d'un driver de système de fichiers . . . 11
2.10 Exemples de callbacks pour l'ouverture et la lecture de fichiers . . . 12
3.1 Enregistrement des opérations sur fichier par un driver . . . 18
3.2 Récupération des opérations sur fichier dans l'appel système `open` . . . 19
3.3 Implémentation de `mmap` par le PL111 . . . 24
3.4 Implémentation de l'opération `ioctl` par le PL111 . . . 26
3.5 Initialisation commune du driver du PL050 . . . 27
3.6 Écriture du registre KMIDATA du PL050 . . . 28
3.7 Récupération d'un paquet PS2 envoyé par le clavier . . . 29
3.8 Analyse d'un paquet PS2 envoyé par la souris . . . 29
3.9 Analyse d'un paquet PS2 envoyé par le clavier . . . 30
4.1 Makefile du backend et frontend vfb . . . 36
4.2 Déclaration de la variable Kconfig pour le backend vfb . . . 36
4.3 Modification des fichiers DTS . . . 37
4.4 Description du PL111 dans la DTS de la plateforme Versatile Express . . . 40
4.5 Fonction d'initialisation du backend vfb . . . 41
4.6 Création du chemin `/backend/vfb` dans l'agence . . . 41
4.7 Création du chemin `/devices/<domain-id>/vfb/0` dans le frontend . . . 42
4.8 Création du chemin `resolution` et des propriétés `hor` et `ver` . . . 42
4.9 Frontend vfb : récupération de la résolution stockée dans le VBStore . . . 42
4.10 Frontend vfb : réservation de l'espace mémoire pour le framebuffer . . . 43
4.11 Description du driver virtuel framebuffer dans la DTS de SOO.refso3 . . . 43
4.12 Frontend vfb : octroi de l'accès aux pages du framebuffer . . . 44

4.13 Backend vfb : observation du chemin de la grantref 45
4.14 Frontend vfb : allocation d'un espace mémoire pour le framebuffer 45
4.15 Initialisation de la structure `vfb_domfb` d'une ME 46
4.16 Initialisation de la structure `vfb_domfb` pour l'agence 47
4.17 Backend vfb : libération des ressources lorsqu'une ME est terminée 48
4.18 Backend vinput : structure de la réponse 50
4.19 Backend vinput : envoi d'une réponse au frontend 51
4.20 Backend vinput : envoi d'une réponse au frontend 52
4.21 Frontend vinput : tri des événements clavier–souris 53
4.22 `input.c` : envoi des valeurs minimales et maximales des écrans tactiles vers le backend 56
4.23 Backend vinput : enregistrement des valeurs minimales et maximales des écrans tactiles 57
4.24 Envoi des événements « touch » au driver de la souris 57
4.25 Configuration du `config.txt` pour les écrans tactiles connectés via HDMI . . 58
4.26 Allocation de l'espace mémoire pour le framebuffer de l'agence 59
4.27 Sauvegarde du framebuffer du VideoCore dans celui de l'agence 59
4.28 Thread de copie périodique des framebuffers 60